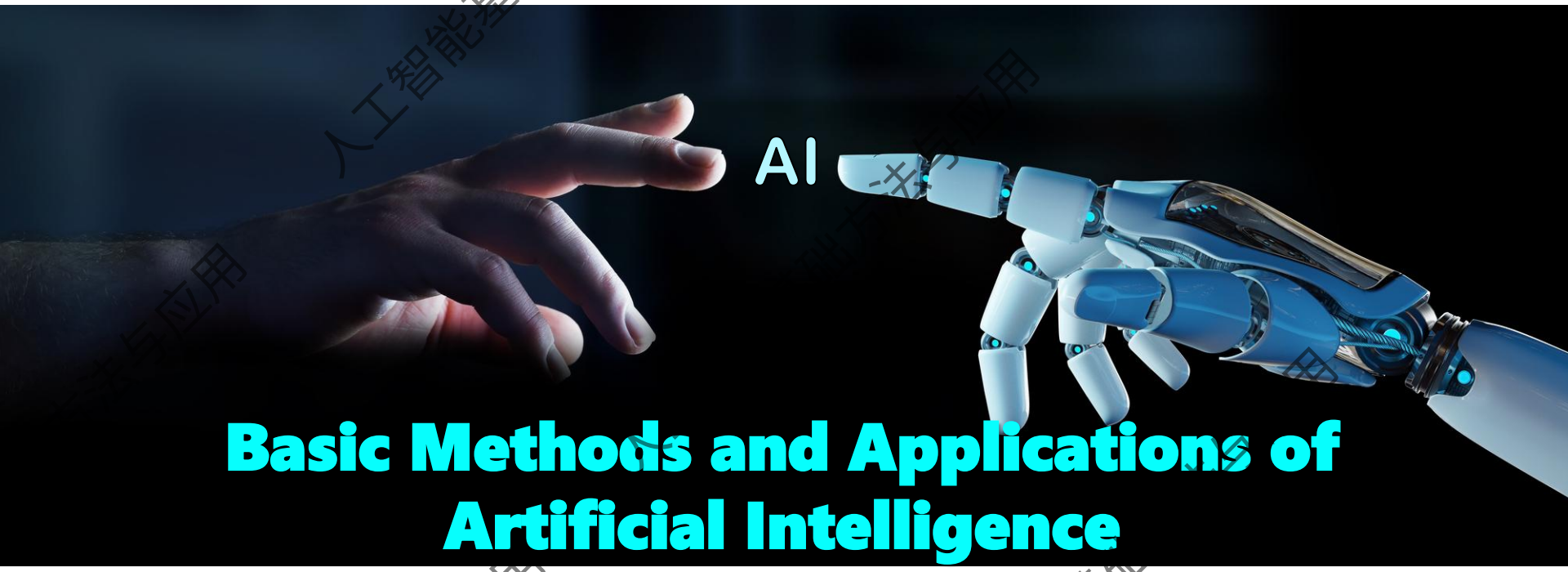


Basic Methods and Applications of Artificial Intelligence

——深度学习

人工智能学院 / 人工智能与机器人研究所
人机混合增强智能全国重点实验室
陈炜煌

IAIR Est. 1986
Institute of Artificial Intelligence and Robotics, XJTU

IAIR Est. 1986

人工智能基础方法与应用

线性回归

候选函数集合

模型

$f_1, f_2 \cdots$

$$L(w,b) = \sum_{n=1}^{10} \left(\hat{y}^n - \left(b + w \cdot x_{cp}^n\right)\right)^2$$

用损失函数评价模型f的好坏

选择“最佳”函数

通过梯度下降寻找最优参数

训练数据

$$f^* = arg\min_f L(f)$$

$$w^*, b^* = arg\min_{w,b} L(w,b)$$

$$= arg\min_{w,b} \sum_{n=1}^{10} \left(\hat{y}^n - \left(b + w \cdot x_{cp}^n\right)\right)^2$$

人工智能基础方法与应用

分类

$$P(x \mid C_i) = \mathcal{N}(x; \mu_i, \Sigma_i)$$

$$f_{\mu,\Sigma}(x) = \frac{1}{(2\pi)^{D/2}} \frac{1}{|\Sigma|^{1/2}} exp\left\{-\frac{1}{2}(x-\mu)^T \Sigma^{-1}(x-\mu)\right\}$$

$$P(C_1|x) = \frac{P(x|C_1)P(C_1)}{P(x|C_1)P(C_1) + P(x|C_2)P(C_2)}$$

分类规则：如果$P(C_1|x) > 0.5$, 预测类别为1

否则, 预测类别为2

候选函数集合

用似然函数评价模型f的好坏

选择“最佳”概率模型

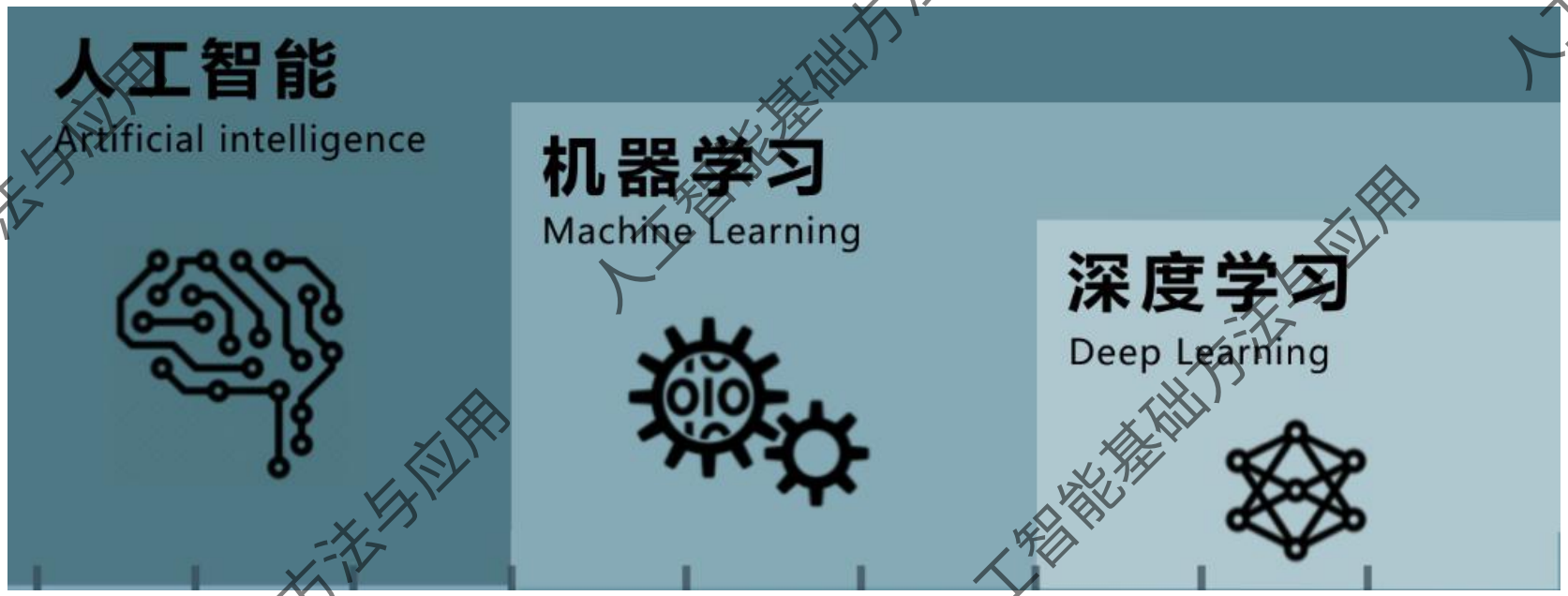
通过最大似然估计，无需梯度下降，直接得到最优参数

$$\mu^*, \Sigma^* = arg \max_{\mu,\Sigma} L(\mu, \Sigma)$$

训练数据

$$\mu^* = \frac{1}{79}\sum_{n=1}^{79} x^n \quad \Sigma^* = \frac{1}{79}\sum_{n=1}^{79} (x^n - \mu^*)(x^n - \mu^*)^T$$

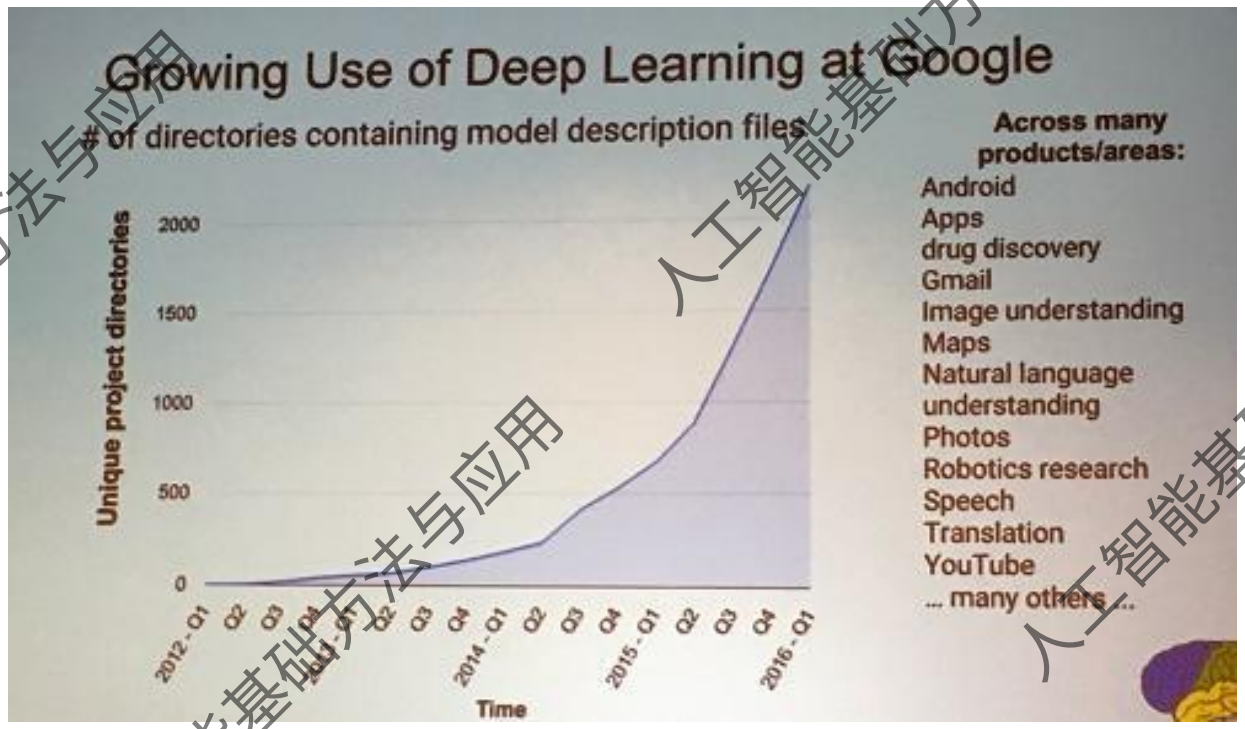
人工智能基础方法与应用



人工智能基础方法与应用

深度学习为何受到关注

- 深度学习在图像、语音、自然语言等领域不断取得突破，并迅速走向大规模实际应用

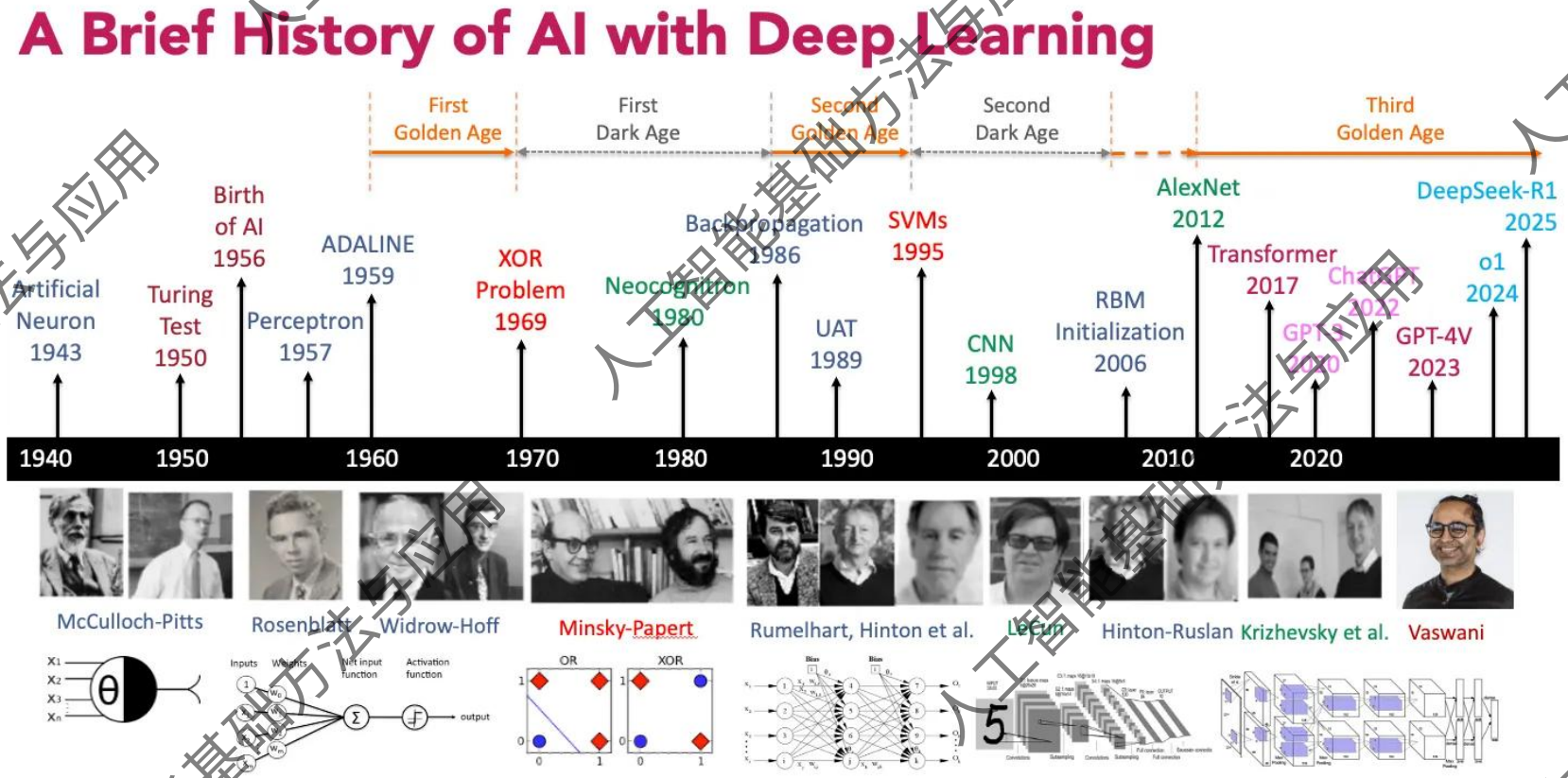


Deep learning trends at Google. Source: SIGMOD/Jeff Dean

- 安卓系统
- 应用程序
- 药物发现
- Gmail 邮箱
- 图像理解
- 地图
- 自然语言理解
- 照片/图像管理
- 机器人研究
- 语音
- 机器翻译
- YouTube 视频平台

人工智能基础方法与应用

深度学习发展史

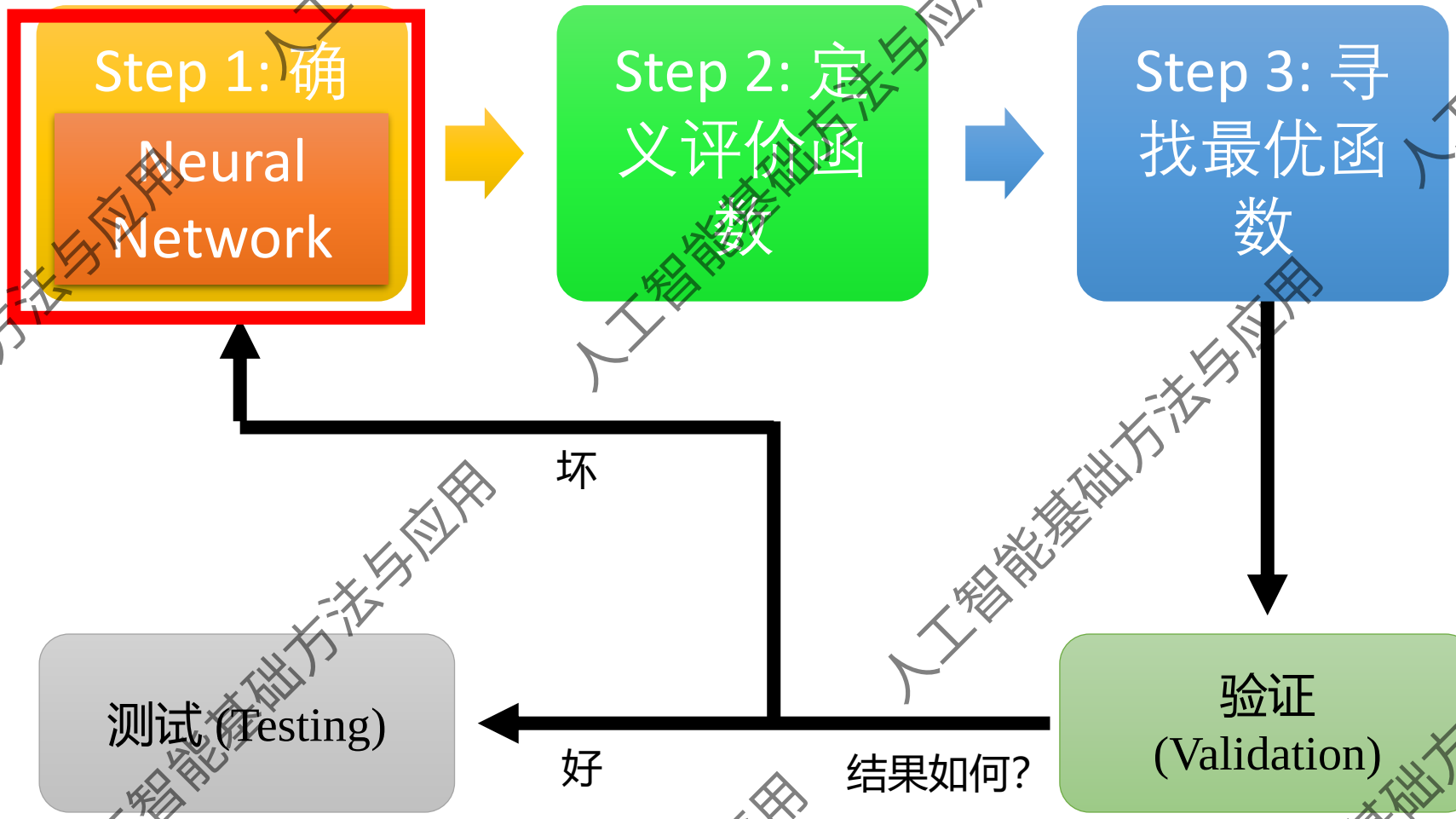


人工智能基础方法与应用

一、深度学习的三个步骤

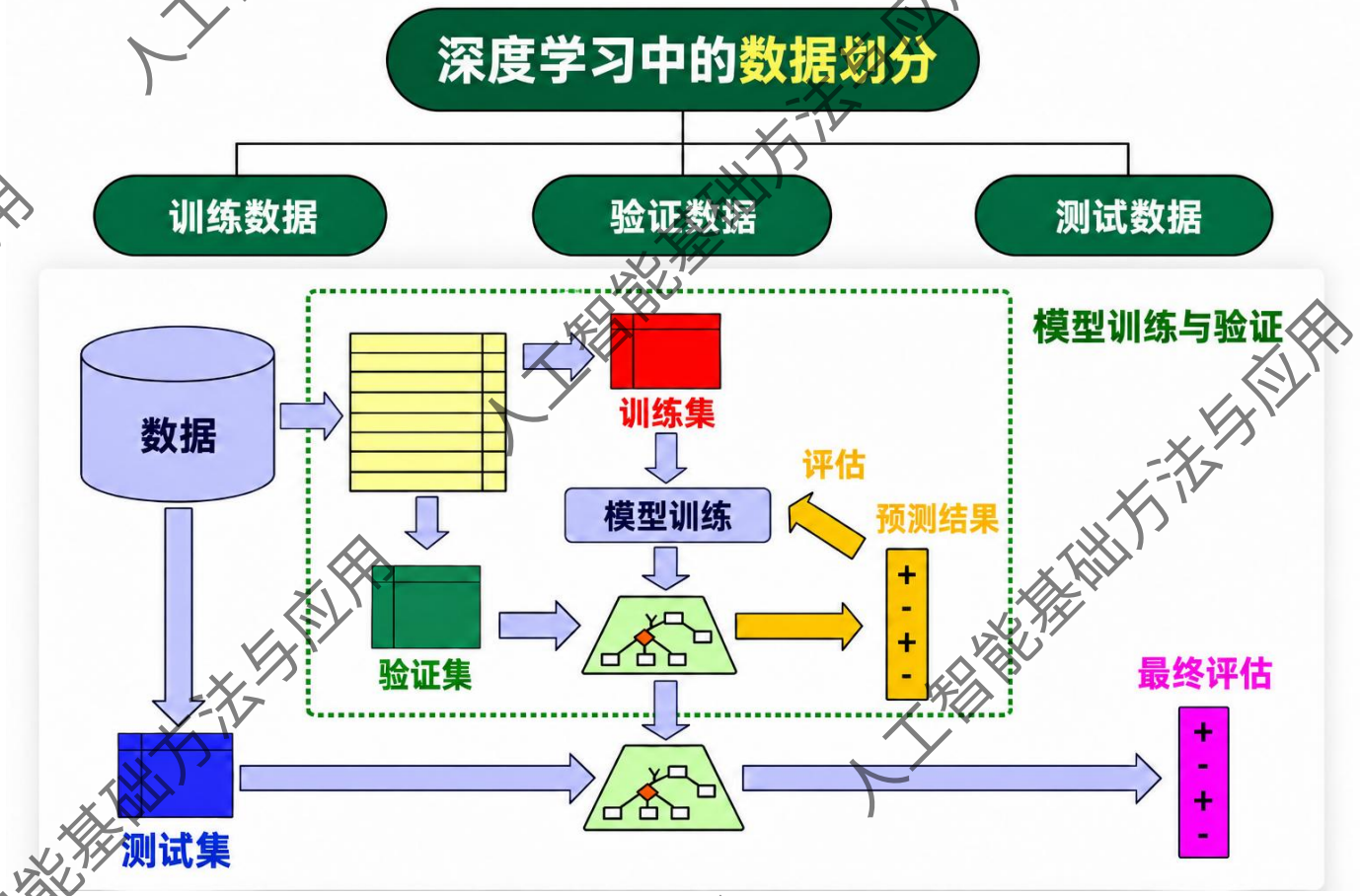
人工智能基础方法与应用

深度学习的三个步骤



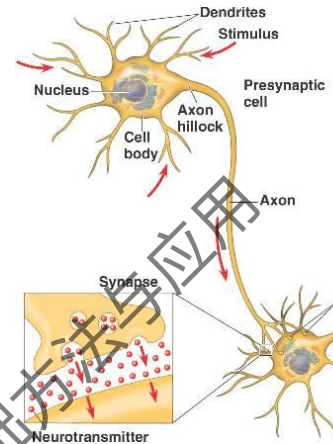
人工智能基础方法与应用

数据集拆分

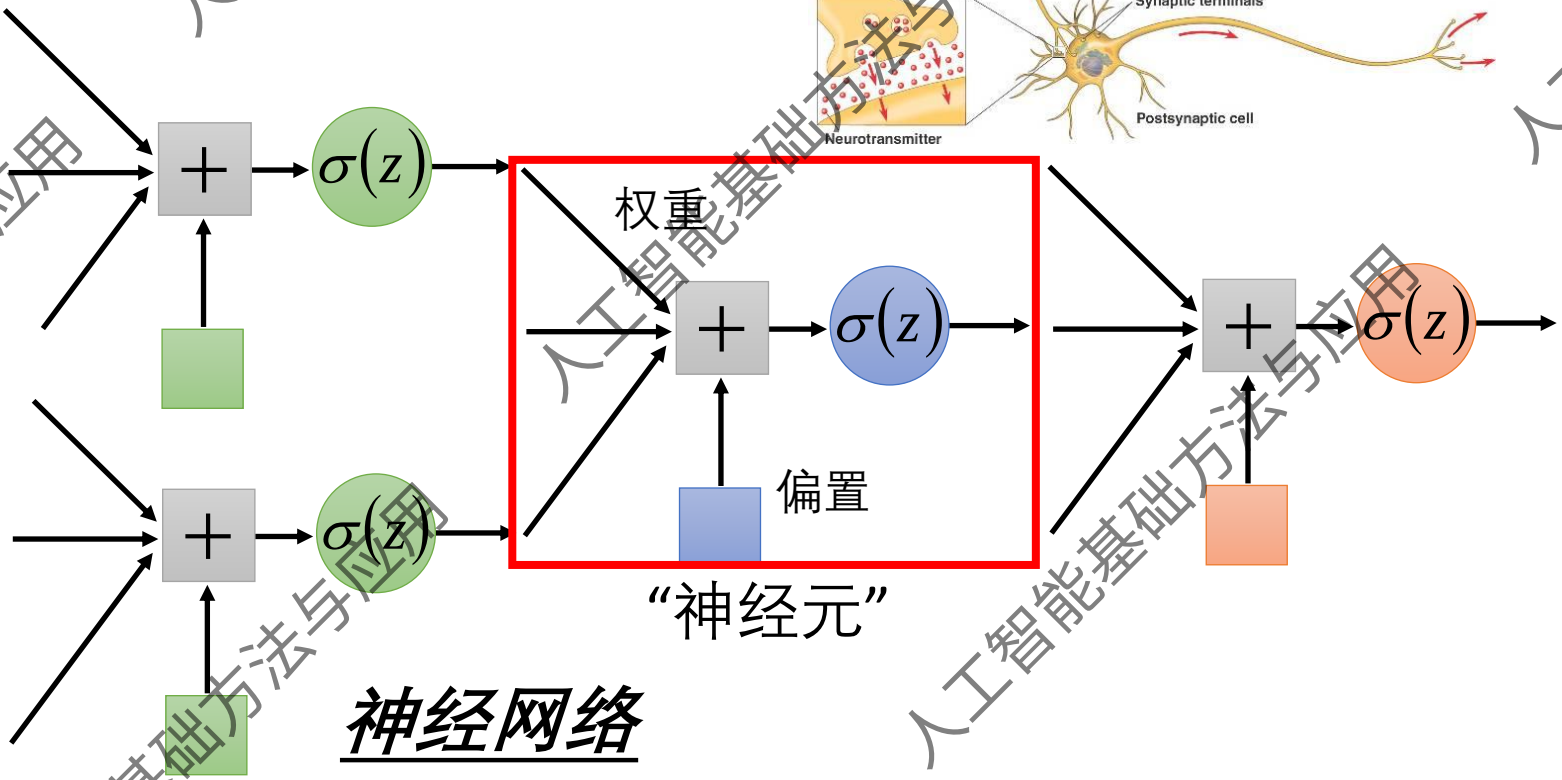


人工智能基础方法与应用

神经网络







神经网络

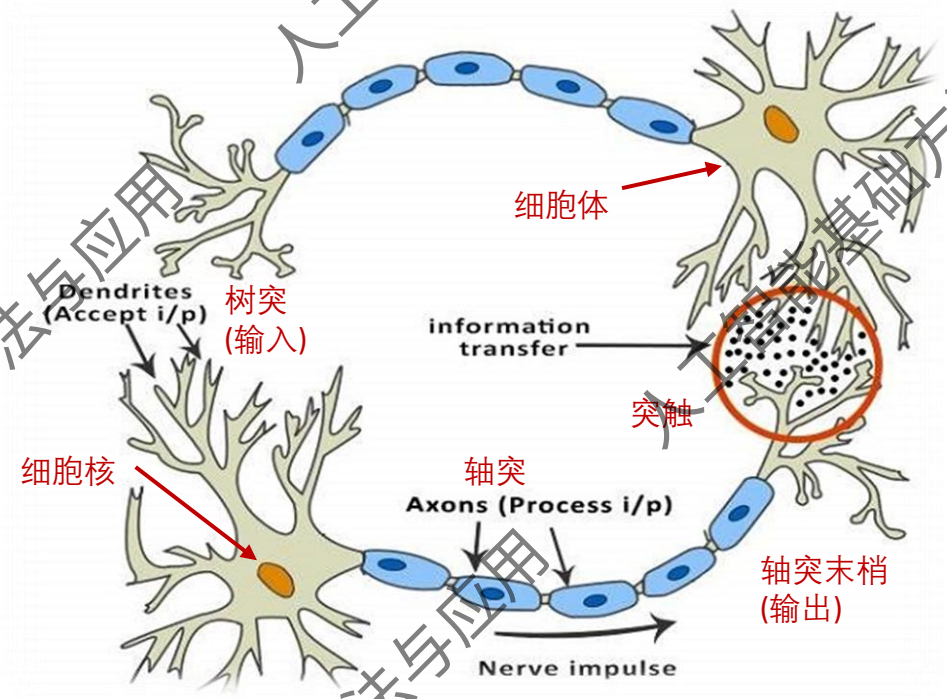
不同的神经元连接形成的**网络**结构

网络参数θ: 网络中所有权重和偏置的集合

人工智能基础方法与应用

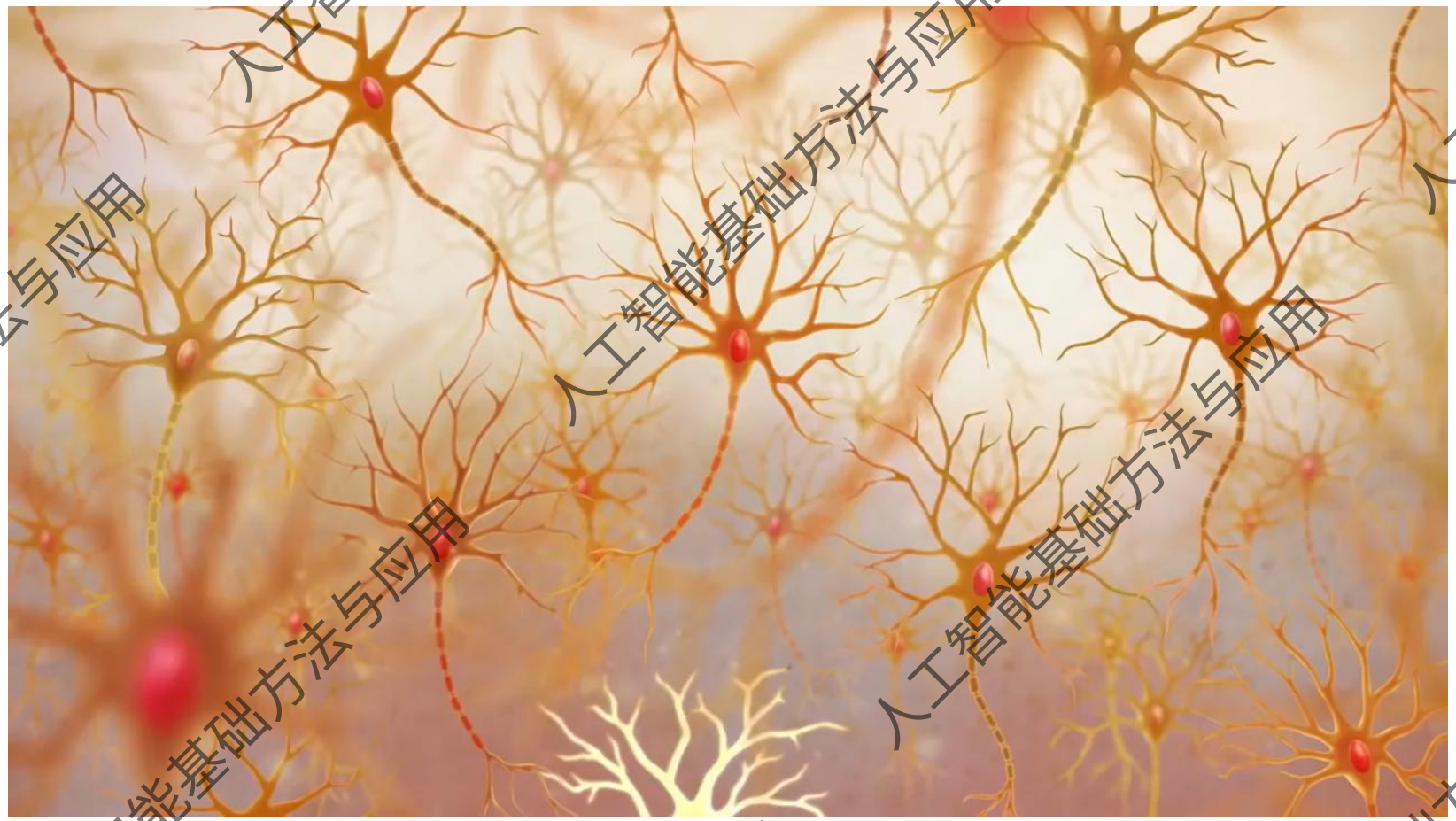
神经元（Neuron）

- 神经元的基本结构：
  - **细胞体**：神经细胞的主体部分，由细胞核、细胞质、细胞膜等组成
  - **树突**：细胞体向外伸出的众多较短的神经纤维分枝，用以**接受来自其他神经元的信号**
  - **轴突**：细胞体向外伸出的最长的一条神经纤维分枝，用以**输出信号**



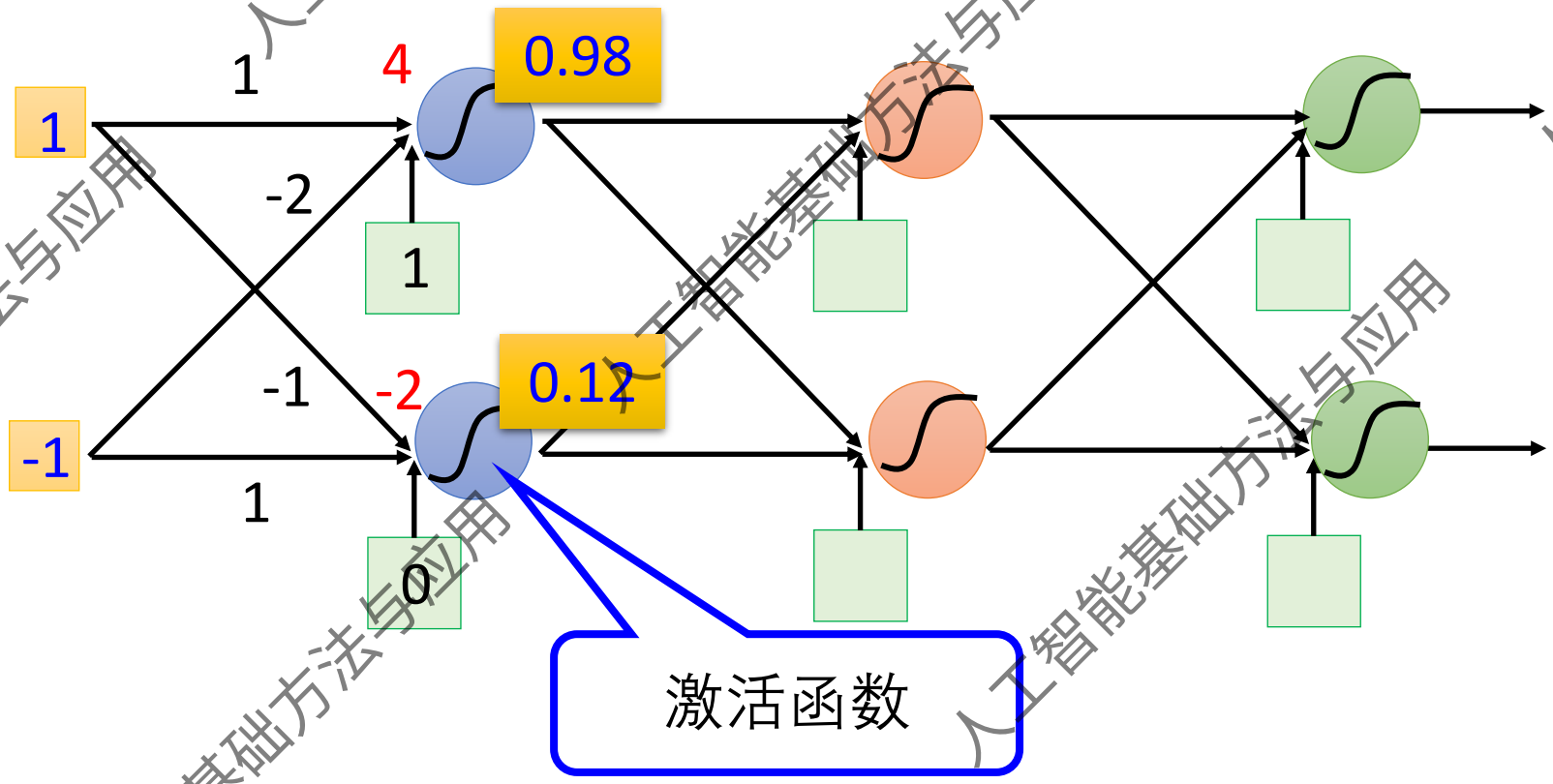
人工智能基础方法与应用

神经元及其网络



人工智能基础方法与应用

全连接前馈网络：第一层计算



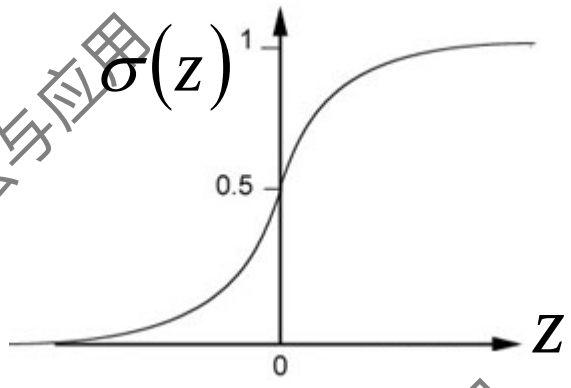
全连接”（Fully Connected）：描述神经网络**层与层之间的连接方式**，如果前一层有m个神经元，后一层有n个神经元，那么全连接表示**前一层的每一个神经元，都和后一层的每一个神经元相连接**

激活函数

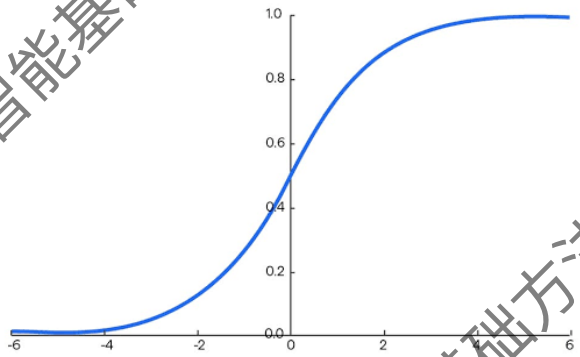
- 作用于神经元线性组合结果上的非线性映射函数，用于产生该神经元的输出
- 核心作用：引入非线性，帮助神经网络学习数据中的复杂模式
- 激活函数没有引入额外的参数

Sigmoid函数

$$\sigma(z)=\frac{1}{1+e^{-z}}$$

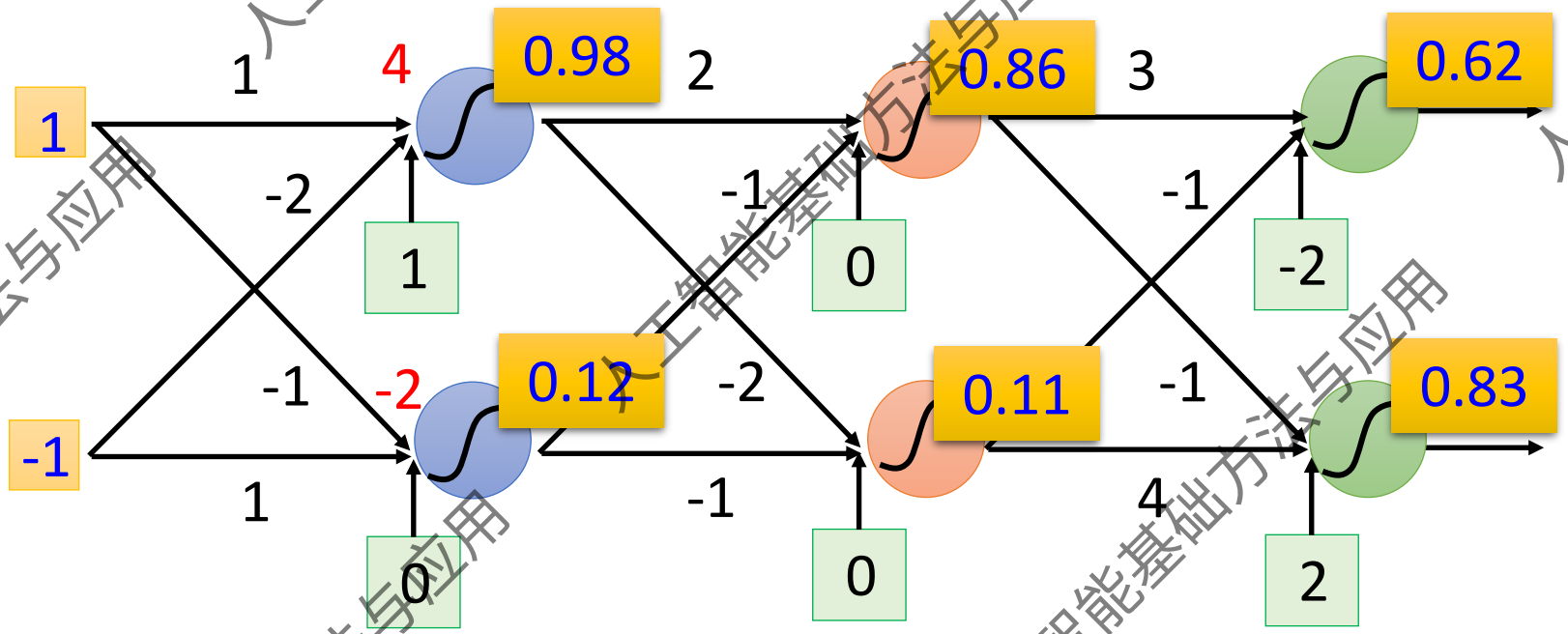


Softmax Function



$$\alpha_i' = exp(\alpha_i)/\sum_j exp(\alpha_j)$$

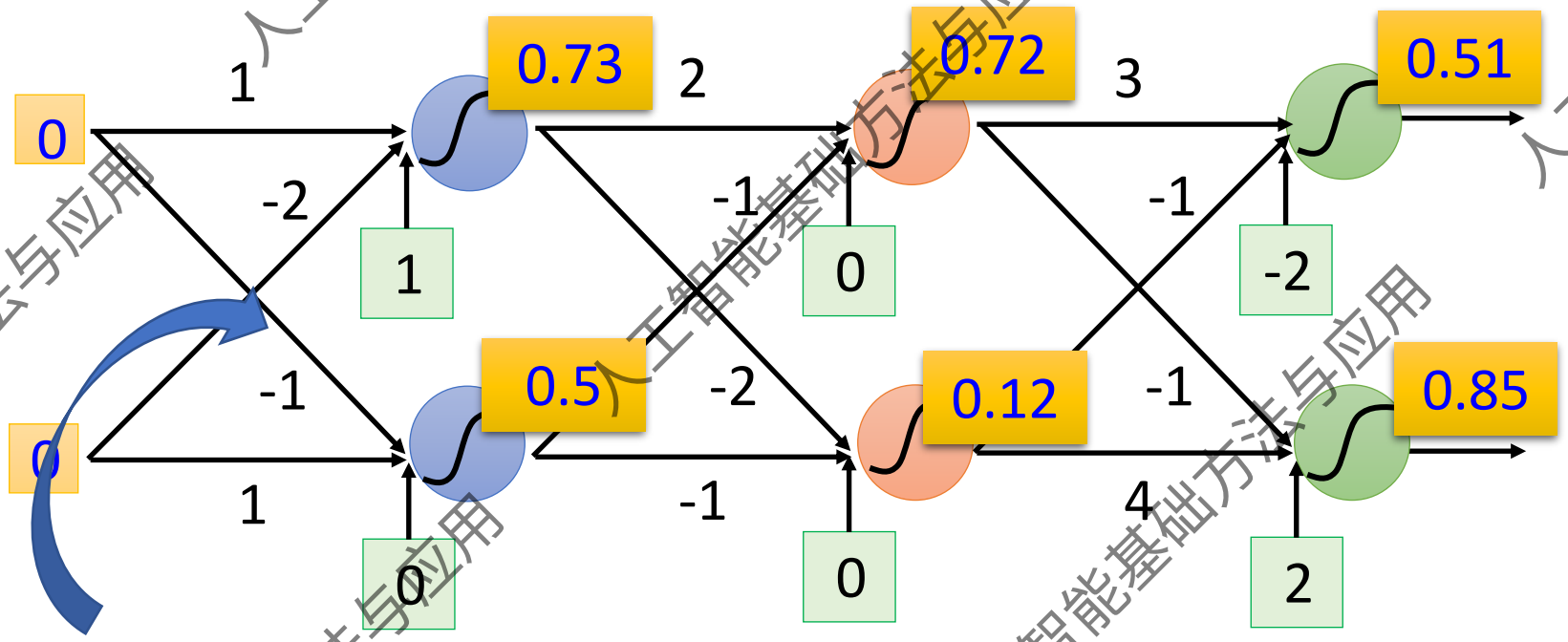
神经网络的前向传播



在全连接网络中，每一层的输出都可以看作一个新的**特征向量**；随着网络逐层向前传播，原始输入被不断转换成更适合当前任务的**特征表示**

人工智能基础方法与应用

神经网络就是函数集合



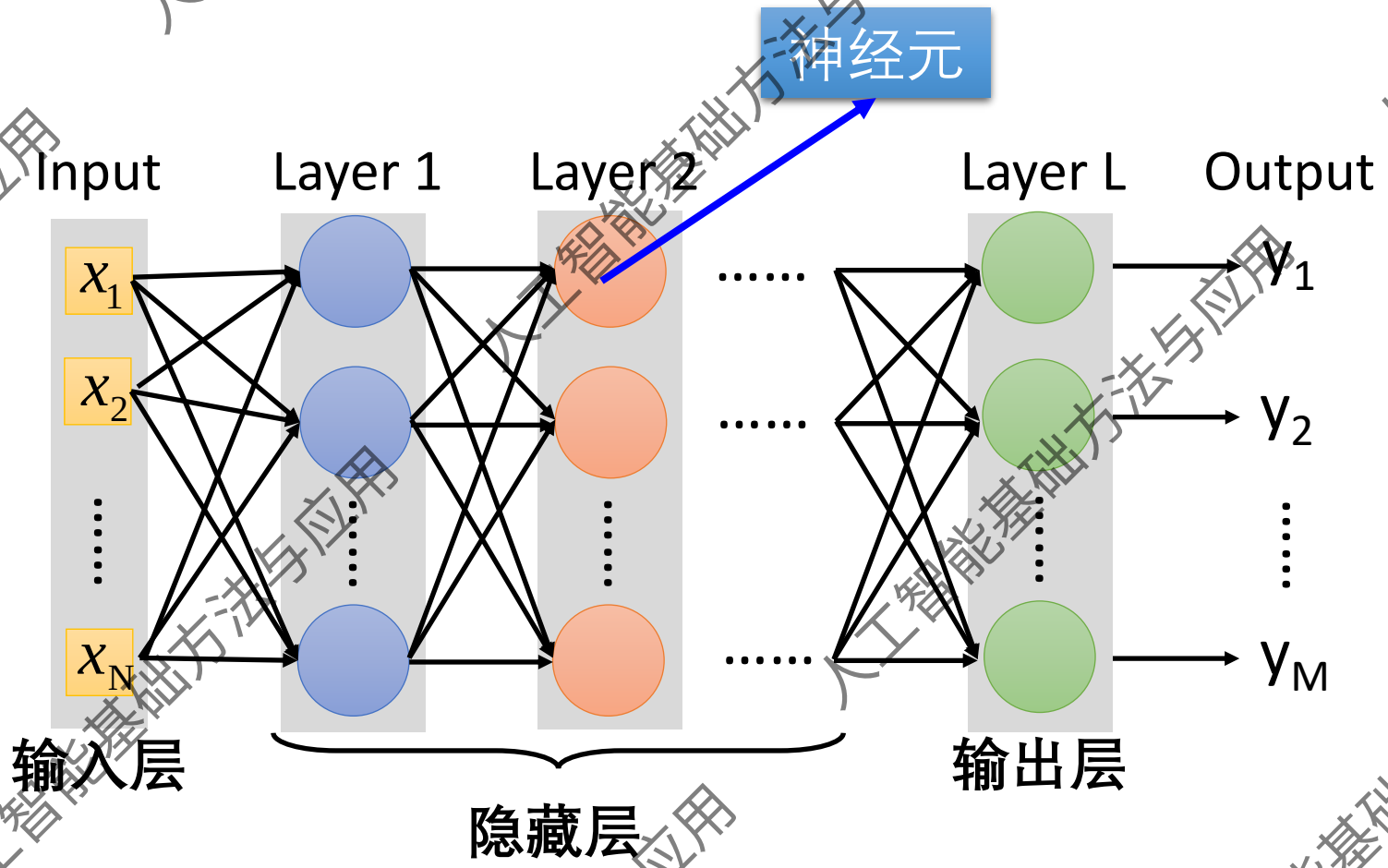
整个网络就是一个函数集合

输入向量，输出向量

$$f\left(\begin{bmatrix}1\\-1\end{bmatrix}\right)=\begin{bmatrix}0.62\\0.83\end{bmatrix} \quad f\left(\begin{bmatrix}0\\0\end{bmatrix}\right)=\begin{bmatrix}0.51\\0.85\end{bmatrix}$$

给定神经网络结构，就确定了*一个函数集合*

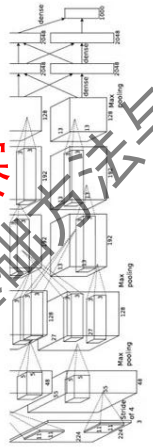
输入层、隐藏层与输出层



既不是直接观测到的输入，也不是任务要求的最终输出，而是网络在计算过程中形成的内部表示

Deep = 隐藏层很多

8 layers



19 layers



22 layers



ImageNet竞赛
错误率

16.4%

7.3%

6.7%

AlexNet (2012)

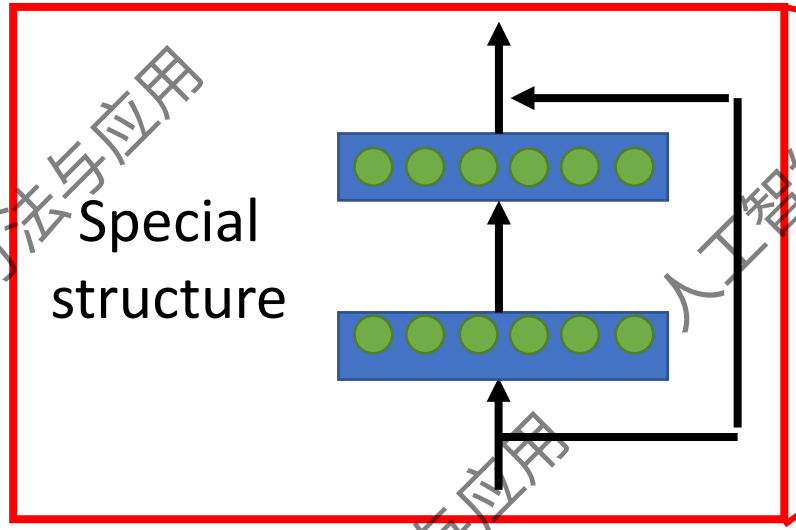
VGG (2014)

GoogleNet (2014)

人工智能基础方法与应用

Deep = 隐藏层很多

ResNet-18, ResNet-34, ResNet-50, ResNet-101, ResNet-152



152 layers

127 layers



16.4%

AlexNet
(2012)

7.3%



VGG
(2014)

6.7%



GoogleNet
(2014)

3.57%

Residual Net
(2015)

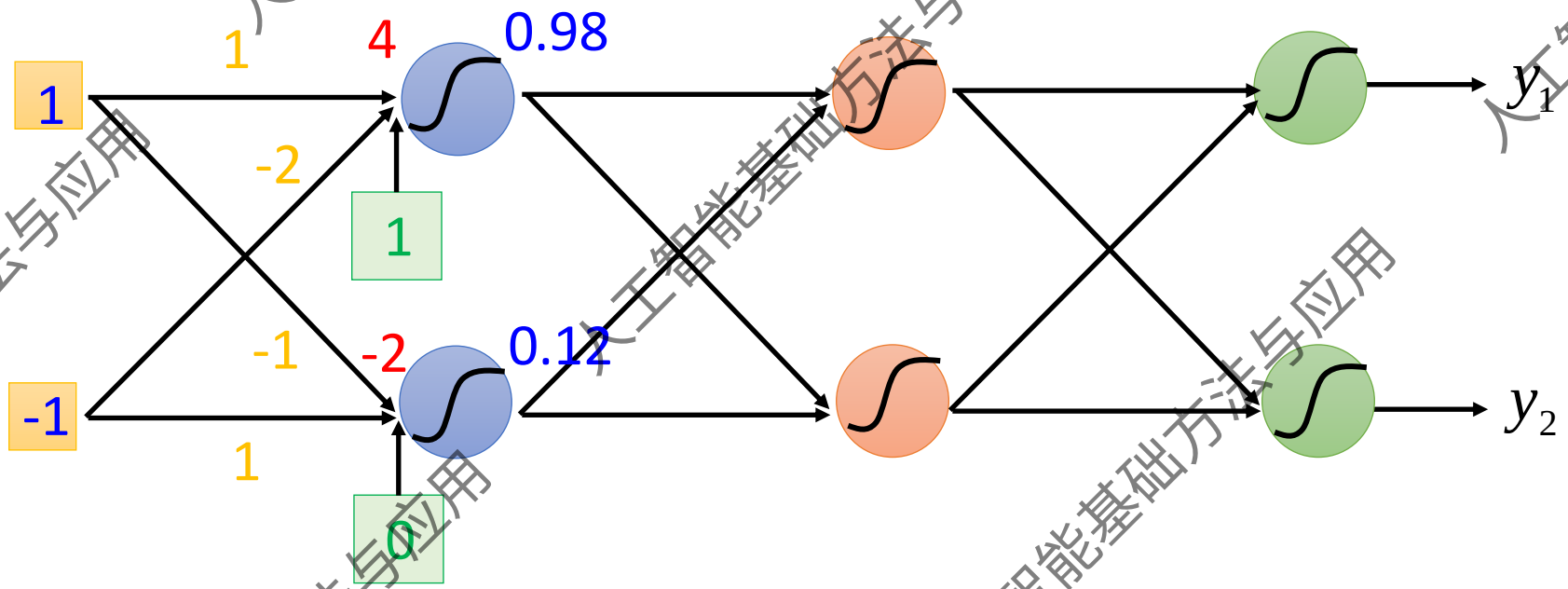
上海中心
大厦

人工智能基础方法与应用

ImageNet大规模视觉识别挑战



用矩阵形式表示神经元运算

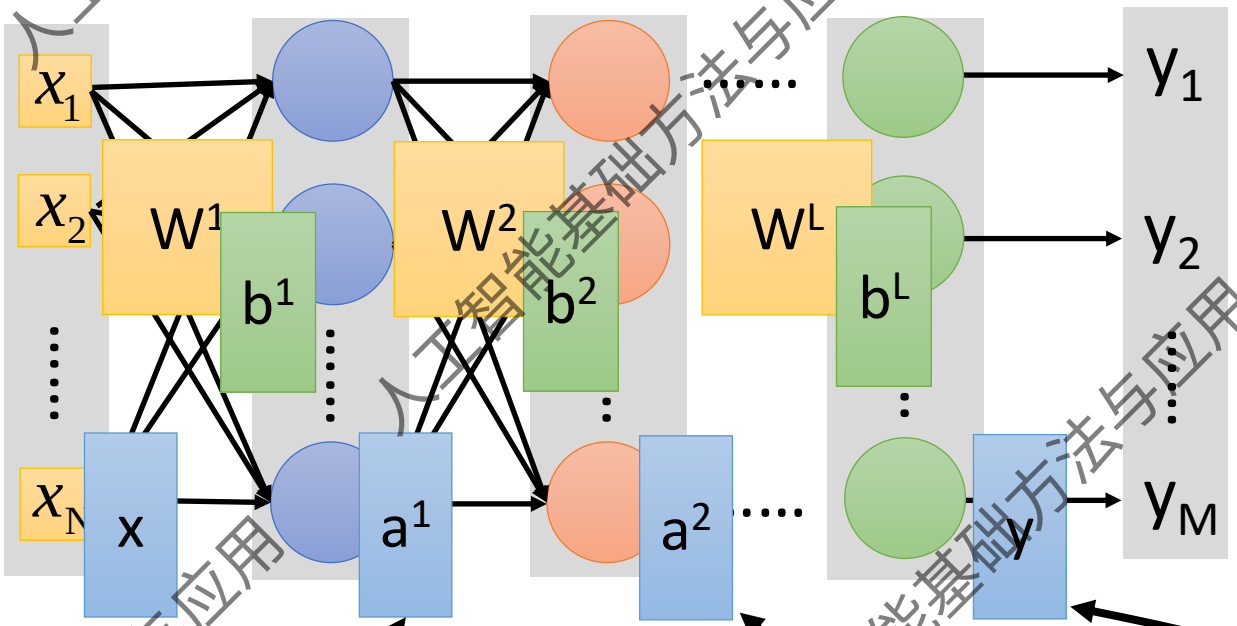


$$\sigma\left(\begin{bmatrix}1 & -2\\ -1 & 1\end{bmatrix}\begin{bmatrix}1\\ -1\end{bmatrix}+\begin{bmatrix}1\\ 0\end{bmatrix}\right)=\begin{bmatrix}0.98\\ 0.12\end{bmatrix}$$

$$\begin{bmatrix}4\\ -2\end{bmatrix}$$

人工智能基础方法与应用

多层网络的矩阵递推



$$\sigma(W^1 x + b^1)$$

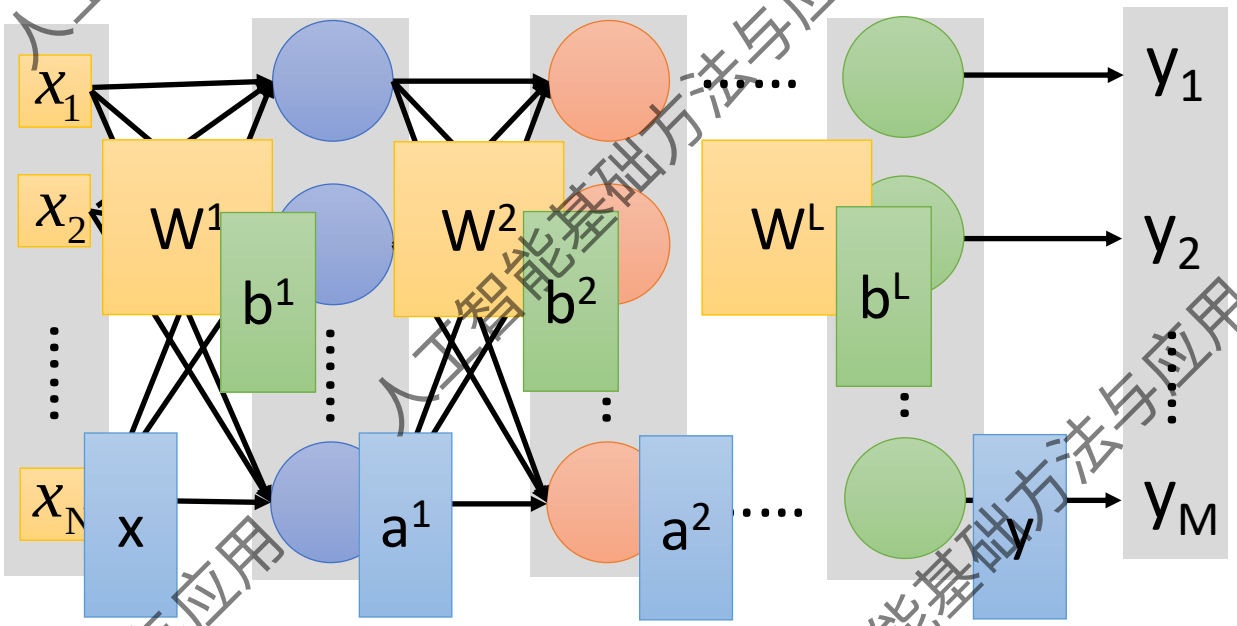
$$\sigma(W^2 a^1 + b^2)$$

$$\sigma(W^L a^{L-1} + b^L)$$

矩阵W的行数等于本层神经元数，列数等于输入维度

人工智能基础方法与应用

复合函数与并行计算



$$y = f(x)$$

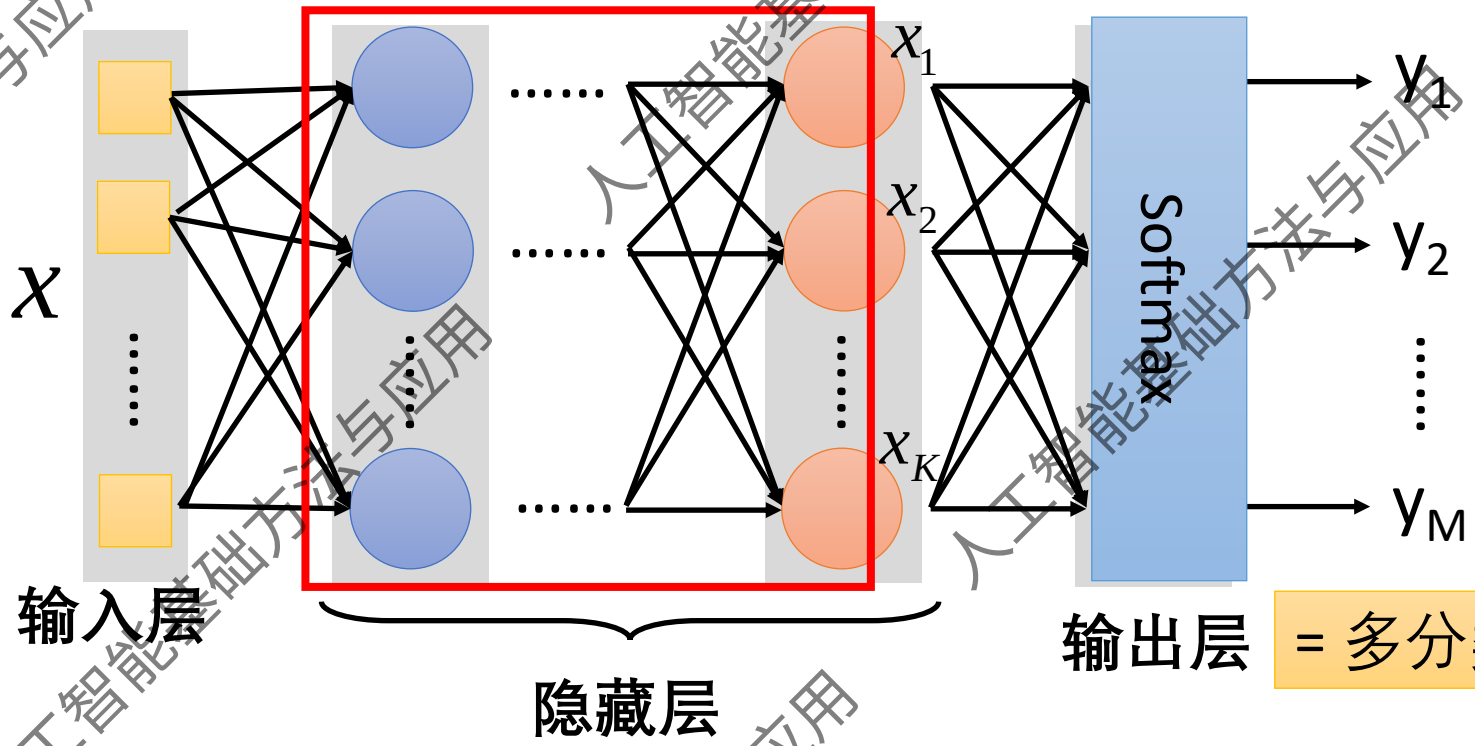
使用并行计算技术加速矩阵运算

$$= \sigma(W^L \cdots \sigma(W^2 \sigma(W^1 x + b^1) + b^2) \cdots + b^L)$$

人工智能基础方法与应用

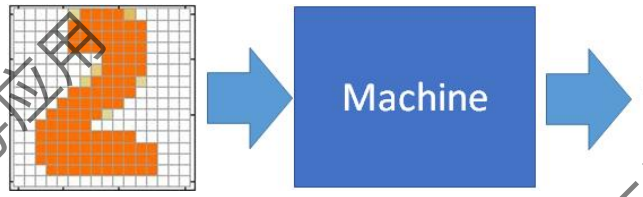
隐藏层提取表示，输出层完成分类

自动特征提取，
替代人工特征工程

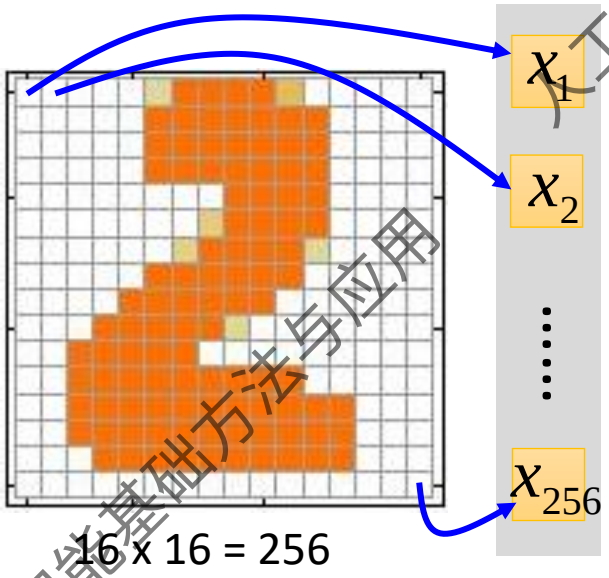


人工智能基础方法与应用

手写数字识别：从图像到概率

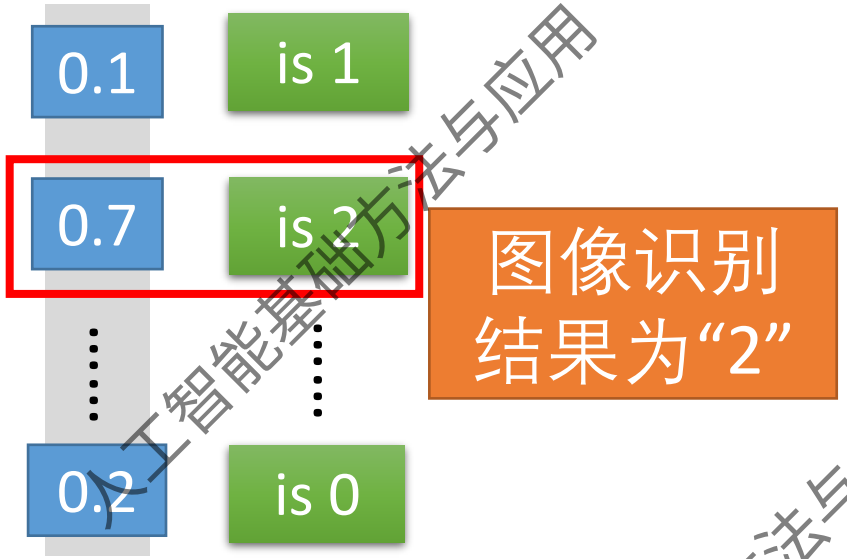


输入



有笔迹 → 1
无笔迹 → 0

输出

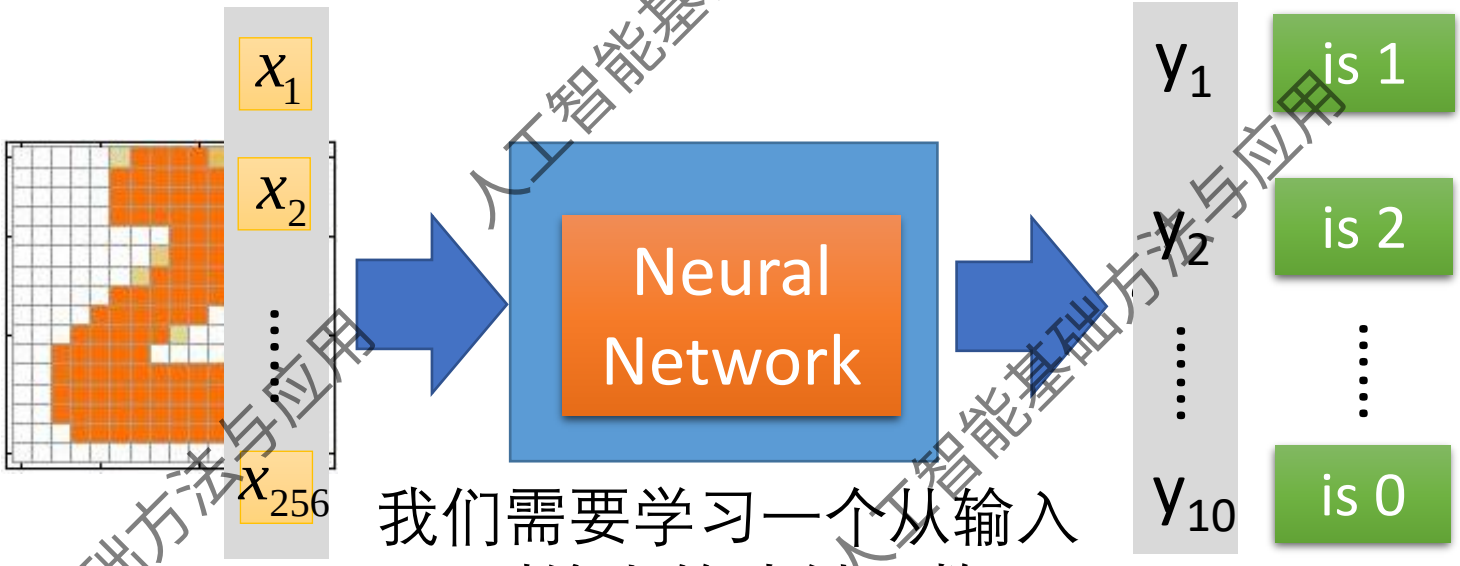


每个输出值表示图像属于对应数字类别的置信度

人工智能基础方法与应用

需要学习一个怎样的函数

- 手写数字识别



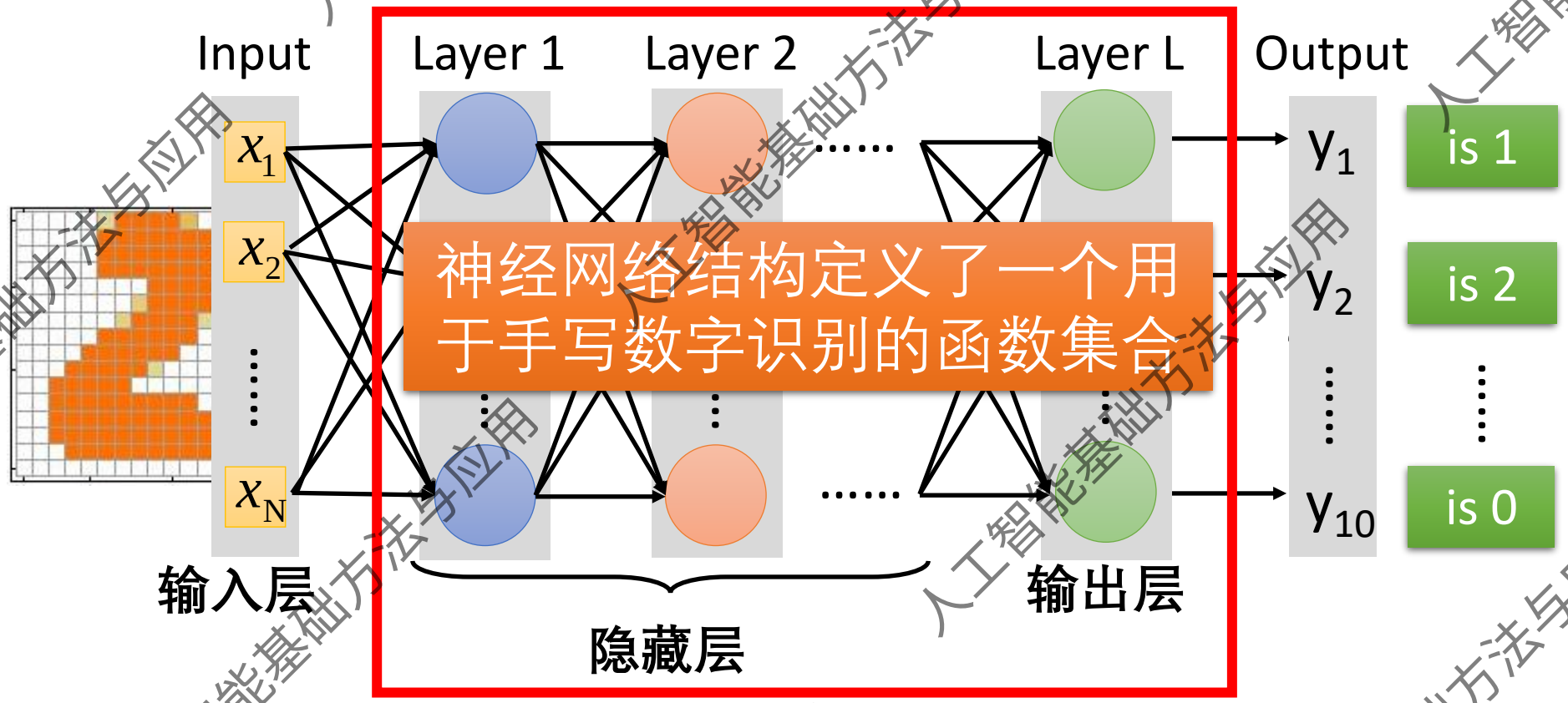
我们需要学习一个从输入到输出的映射函数

输入:
256维向量

输出:
10维向量

人工智能基础方法与应用

网络结构决定候选函数集合



网络结构决定函数集合；好的网络结构更有可能包含适合任务的函数

人工智能基础方法与应用

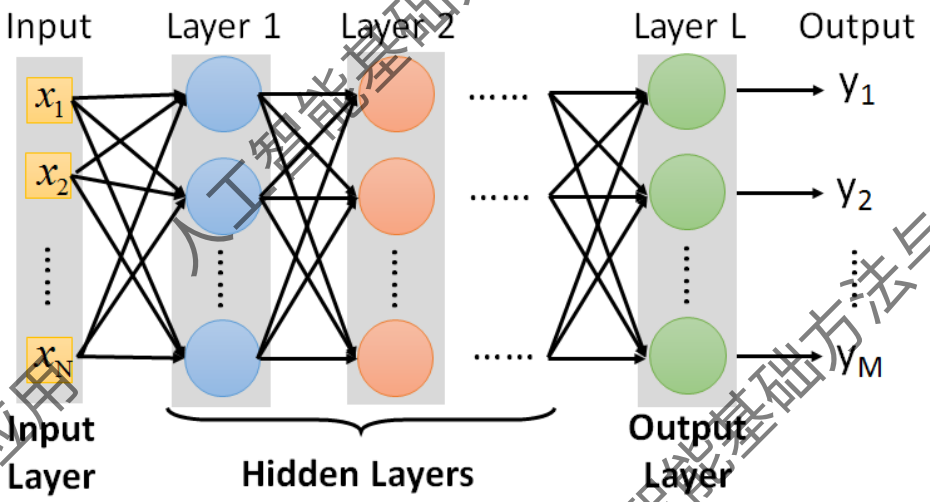
网络结构常见问题

- Q: 网络需要多少层? 每层多少个神经元?

反复试验 + 经验直觉

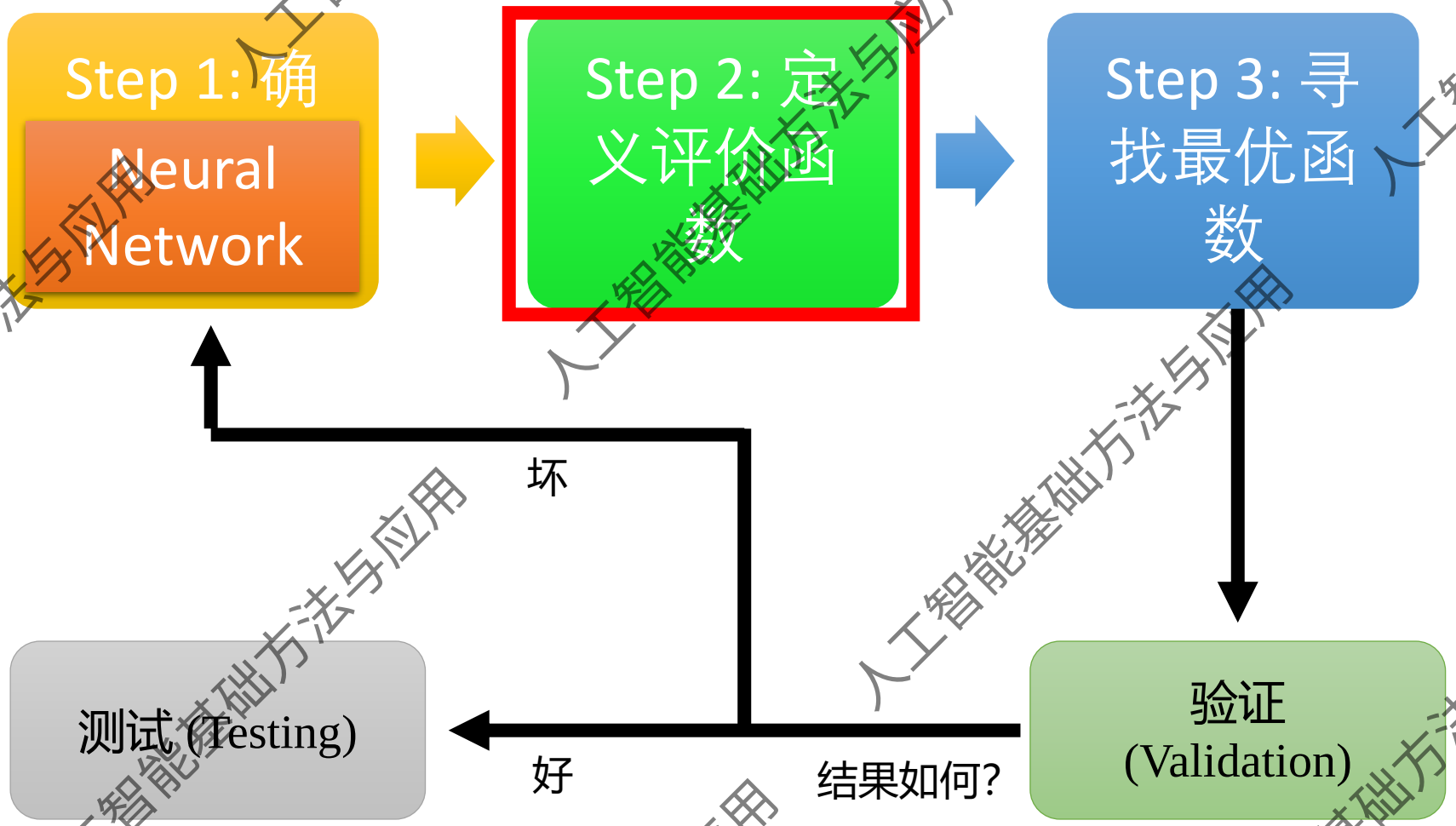
- Q: 网络结构能否自动确定?
  - E.g. 进化人工神经网络
- Q: 我们能否针对任务专门设计网络结构?

卷积神经网络(CNN)



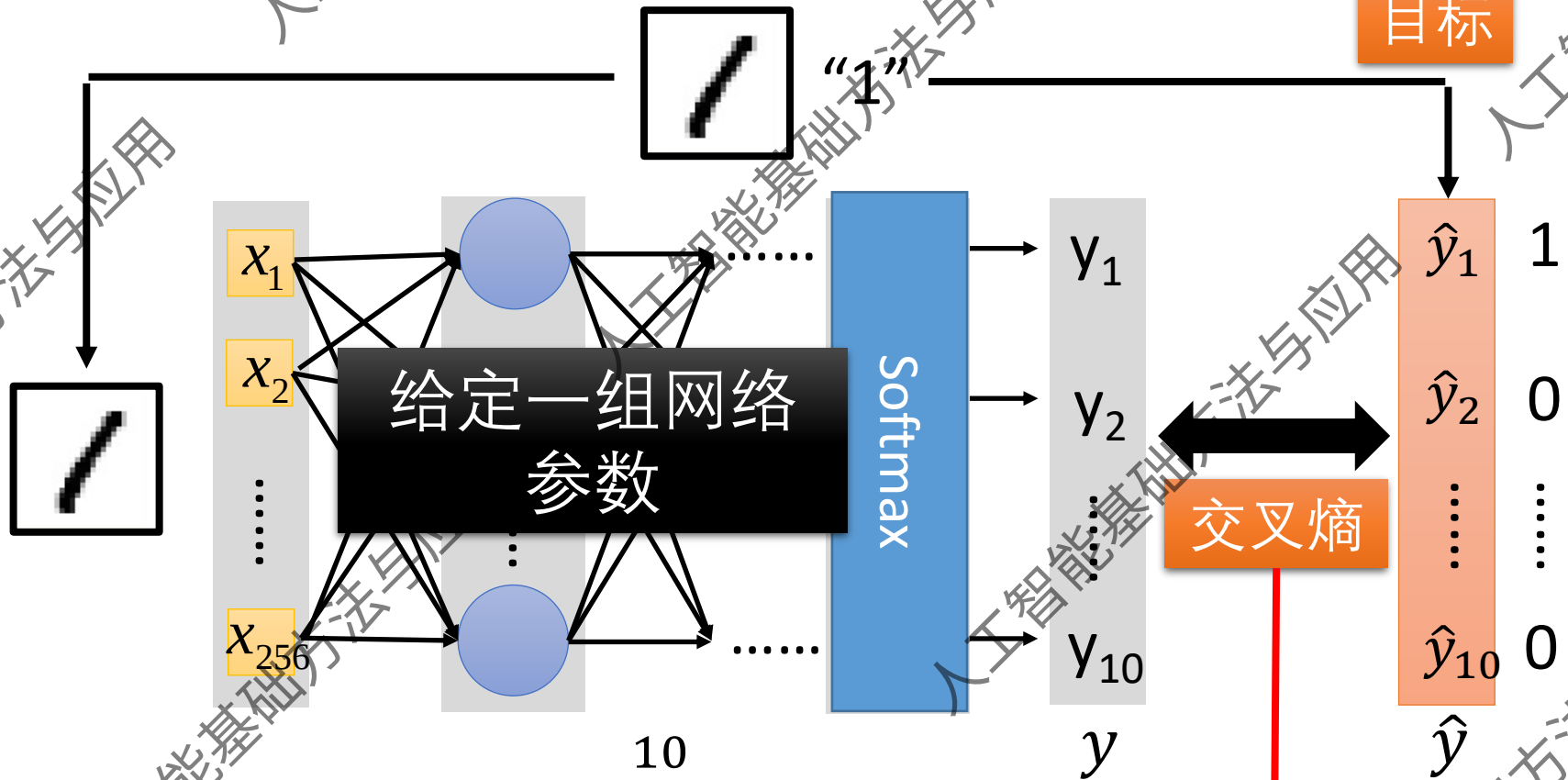
人工智能基础方法与应用

深度学习的三个步骤



人工智能基础方法与应用

单个样本的Softmax交叉熵



$$C(y,\hat{y}) = -\sum_{i=1}^{10} \hat{y}_i ln y_i$$

什么是交叉熵

$$C(y,\hat{y}) = -\sum_{i=1}^{10} \hat{y}_i ln y_i$$

- 假设输入的手写数字真实答案是“1”，那么真实标签采用 one-hot 表示：

$$\hat{y} = [1, 0, 0, \cdots, 0]$$

- 而神经网络经过 Softmax 后可能输出：

$$y = [0.8, 0.05, 0.03, \cdots]$$

- 也就是说，模型认为“是数字1”的概率是 0.8。代入交叉熵：

$$C = -(1 \ln 0.8 + 0 \ln 0.05 + 0 \ln 0.03 + \cdots)$$

- 后面所有乘以 0 的项都消失了，于是：

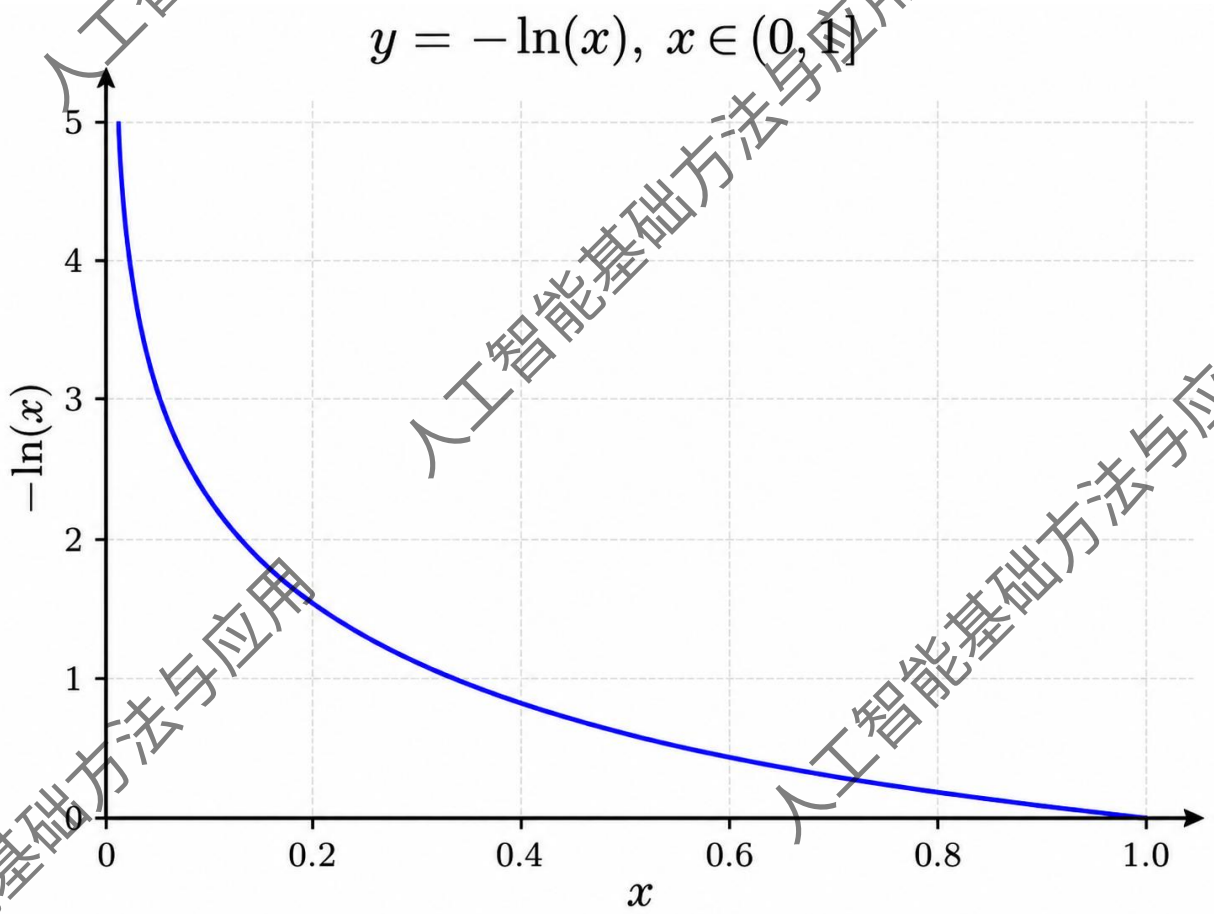
$$\boxed{C = -\ln 0.8}$$

- 所以对于 one-hot 标签，复杂的交叉熵公式实际上可以简单理解成：

$$\boxed{\text{交叉熵损失} = -\ln(\text{正确类别的预测概率})}$$

人工智能基础方法与应用

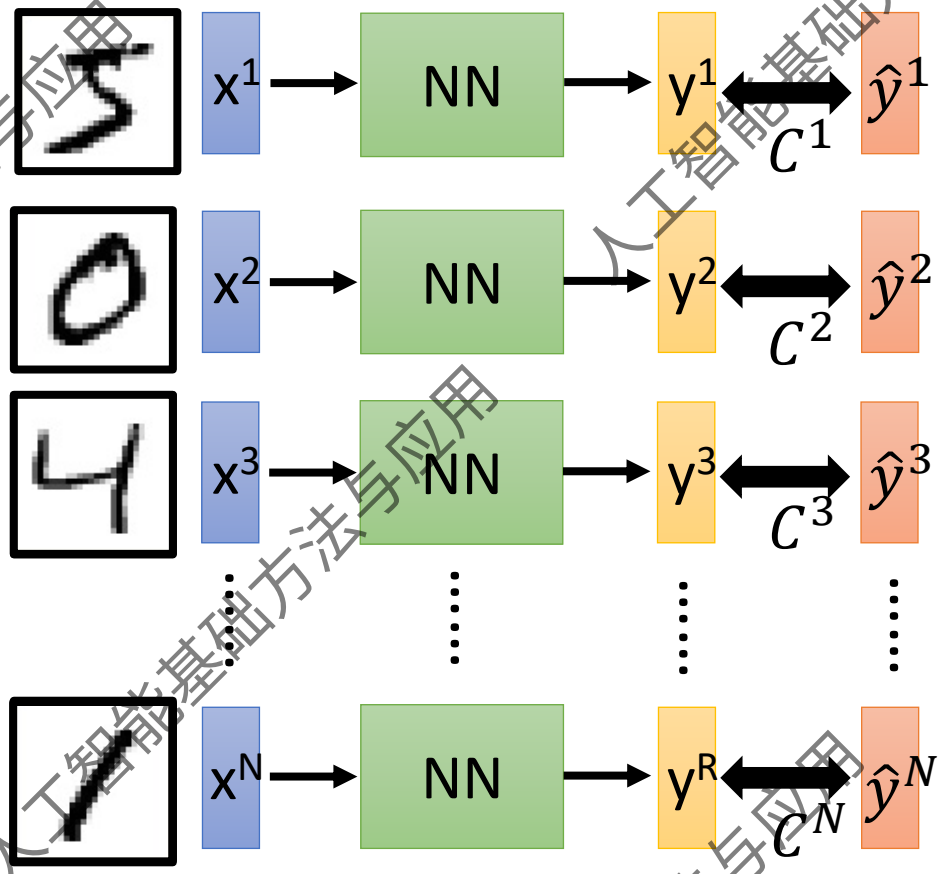
什么是交叉熵



人工智能基础方法与应用

训练集上的整体损失

对于所有的训练数据



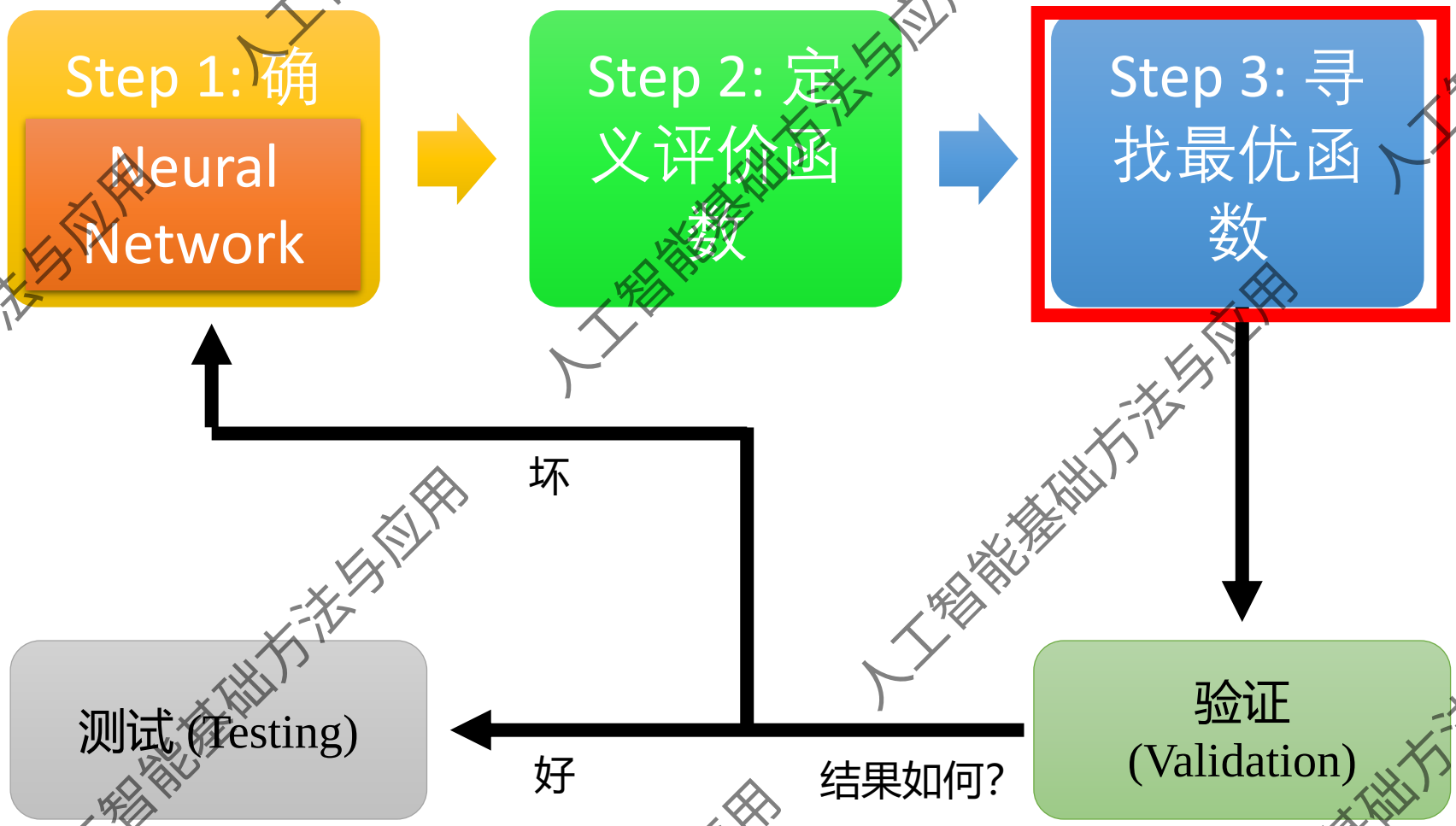
Total Loss:

$$L = \sum_{n=1}^{N} C^n$$

在 ***候选函数集合中寻找一个最优函数*** 能够最小化总损失L

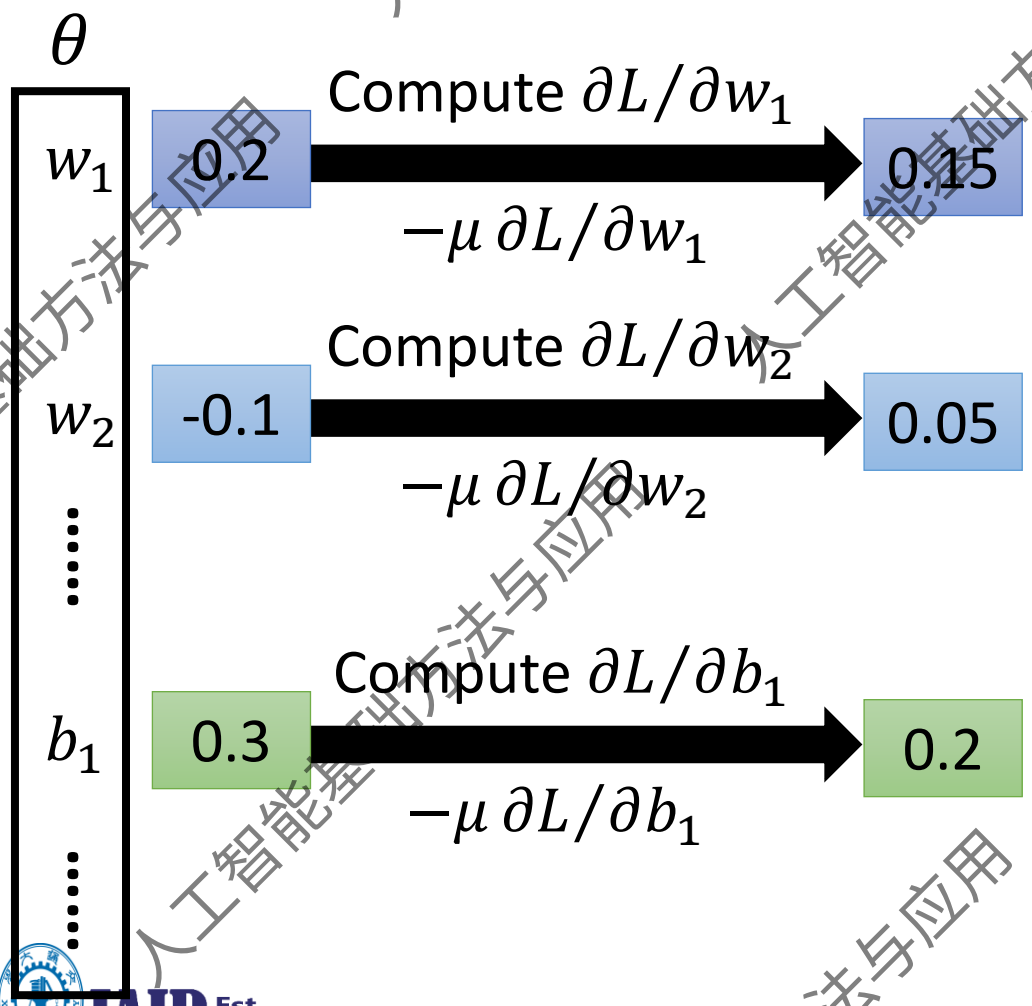
寻找 ***最优神经网络参数θ^**** 能够最小化总损失L

深度学习的三个步骤



人工智能基础方法与应用

梯度下降：完成一次参数更新

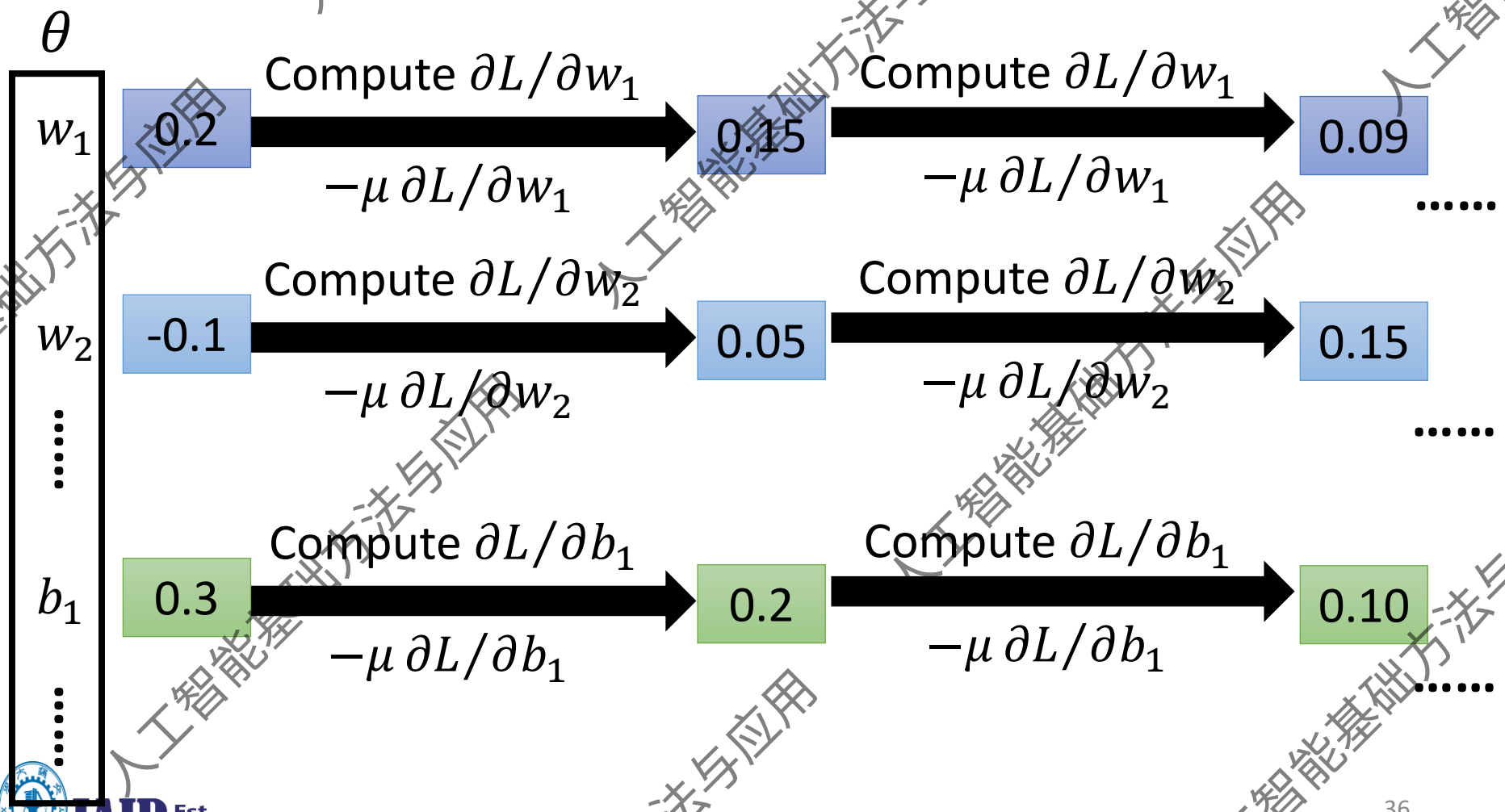


$$\nabla L = \begin{bmatrix} \frac{\partial L}{\partial w_1} \\ \frac{\partial L}{\partial w_2} \\ \vdots \\ \frac{\partial L}{\partial b_1} \\ \vdots \end{bmatrix}$$

gradient

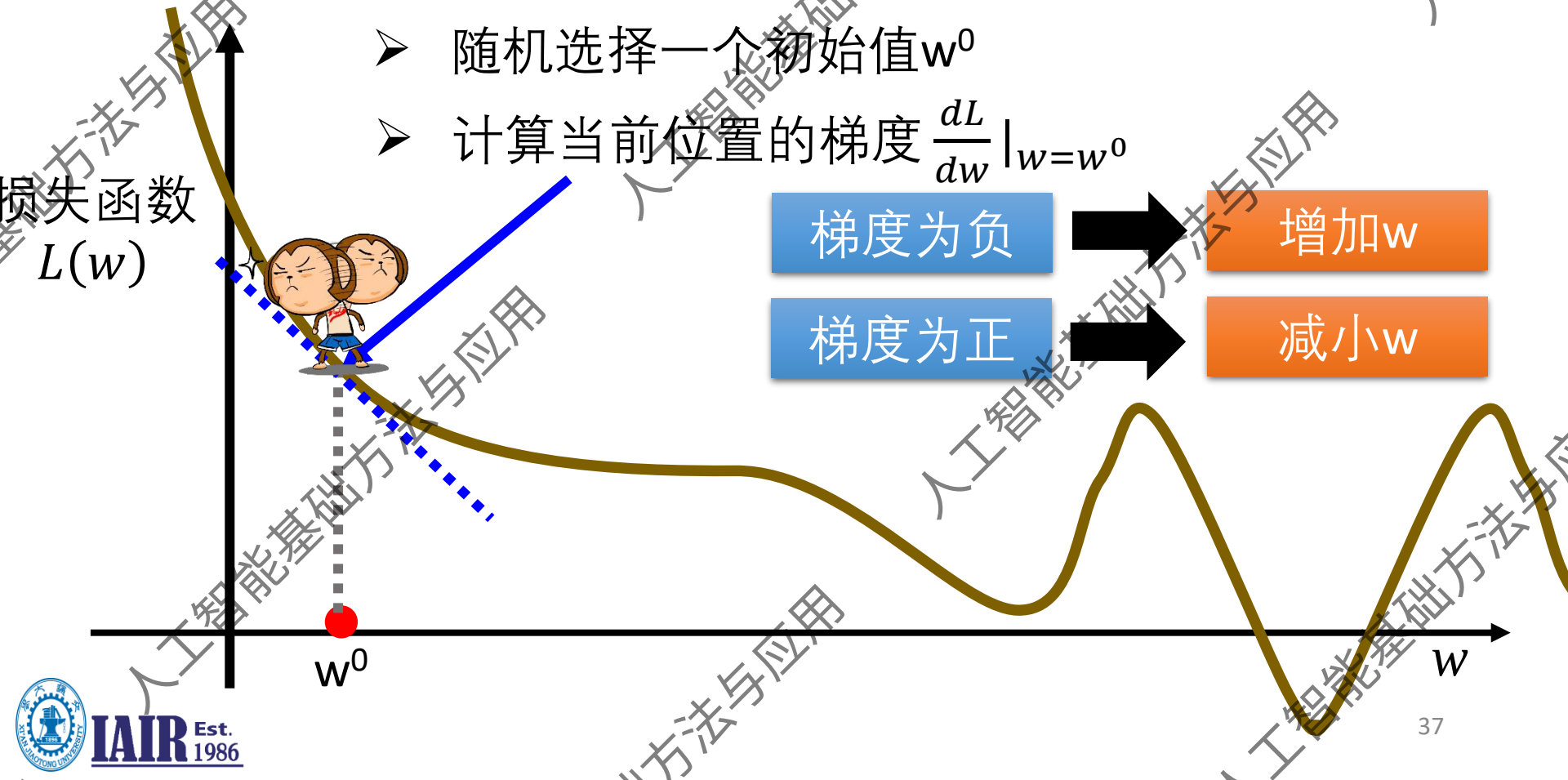
人工智能基础方法与应用

梯度下降：重复迭代


θ
w_1
w_2
b_1
0.2
-0.1
0.3
Compute $\partial L/\partial w_1$
Compute $\partial L/\partial w_2$
Compute $\partial L/\partial b_1$
$-\mu\,\partial L/\partial w_1$
$-\mu\,\partial L/\partial w_2$
$-\mu\,\partial L/\partial b_1$
0.15
0.05
0.2
Compute $\partial L/\partial w_1$
Compute $\partial L/\partial w_2$
Compute $\partial L/\partial b_1$
$-\mu\,\partial L/\partial w_1$
$-\mu\,\partial L/\partial w_2$
$-\mu\,\partial L/\partial b_1$
0.09
0.15
0.10

梯度下降：先判断往哪边走

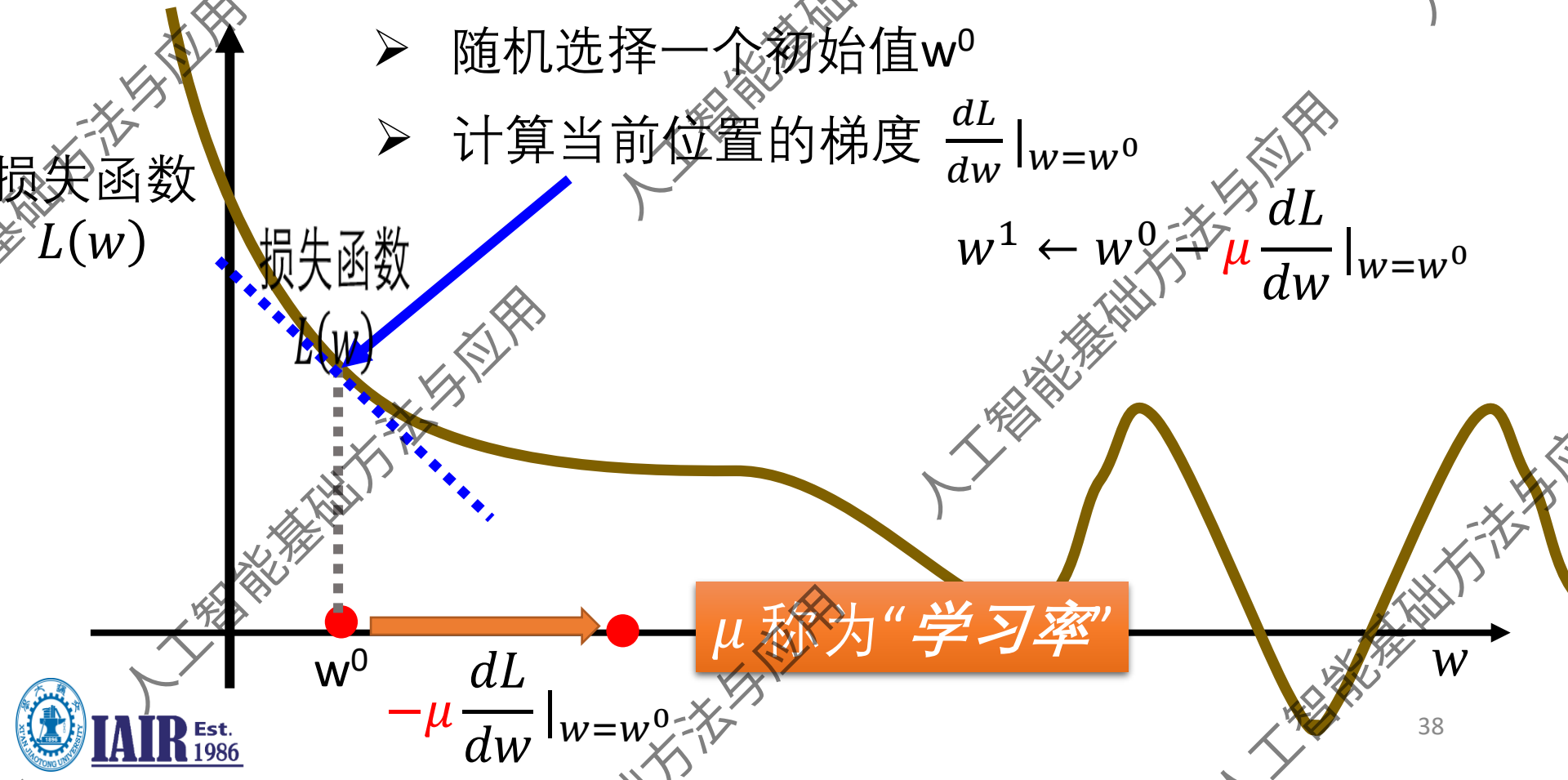
- 考虑只有一个参数w的损失函数$L(w)$：　$w^* = arg\min_w L(w)$



人工智能基础方法与应用

学习率决定每一步的大小

- 考虑只有一个参数w的损失函数$L(w)$：　$w^* = arg\min_w L(w)$



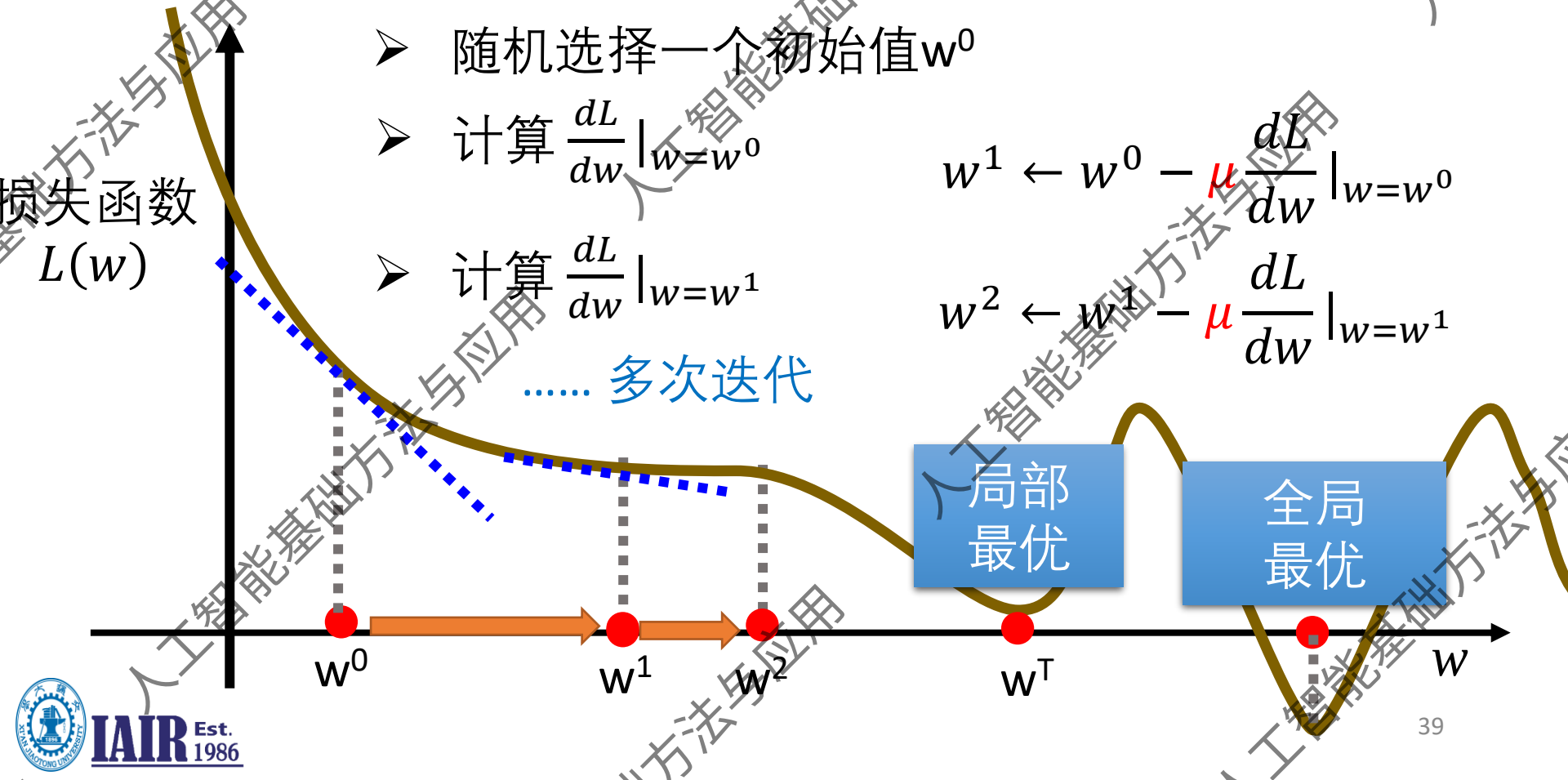
- 随机选择一个初始值w^0
- 计算当前位置的梯度 $\frac{dL}{dw}|_{w=w^0}$

$$w^1 \leftarrow w^0 - \mu \frac{dL}{dw}|_{w=w^0}$$

人工智能基础方法与应用

反复更新直到损失不再明显下降

- 考虑只有一个参数w的损失函数$L(w)$：　$w^* = arg\min_w L(w)$



人工智能基础方法与应用

梯度

$$\nabla L = \begin{bmatrix} \frac{\partial L}{\partial w} \\ \frac{\partial L}{\partial b} \end{bmatrix}$$ gradient

- 两个参数如何计算梯度?

$$w^*, b^* = arg \min_{w,b} L(w, b)$$

- 随机选择初始值w^0, b^0
- 计算 $\frac{\partial L}{\partial w}|_{w=w^0,b=b^0}, \frac{\partial L}{\partial b}|_{w=w^0,b=b^0}$

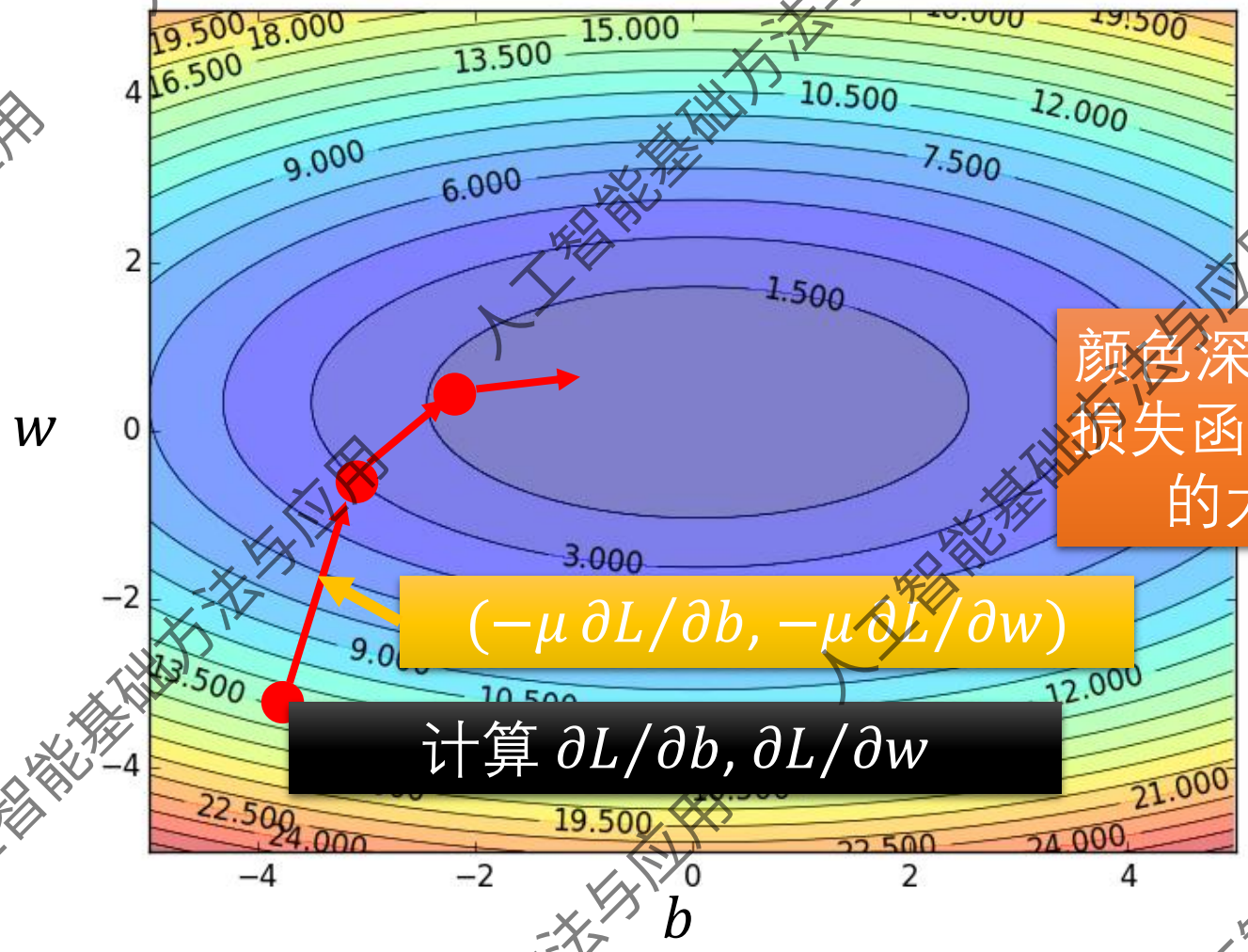
$$w^1 \leftarrow w^0 - \mu \frac{\partial L}{\partial w}|_{w=w^0,b=b^0} \qquad b^1 \leftarrow b^0 - \mu \frac{\partial L}{\partial b}|_{w=w^0,b=b^0}$$

- 计算 $\frac{\partial L}{\partial w}|_{w=w^1,b=b^1}, \frac{\partial L}{\partial b}|_{w=w^1,b=b^1}$

$$w^2 \leftarrow w^1 - \mu \frac{\partial L}{\partial w}|_{w=w^1,b=b^1} \qquad b^2 \leftarrow b^1 - \mu \frac{\partial L}{\partial b}|_{w=w^1,b=b^1}$$

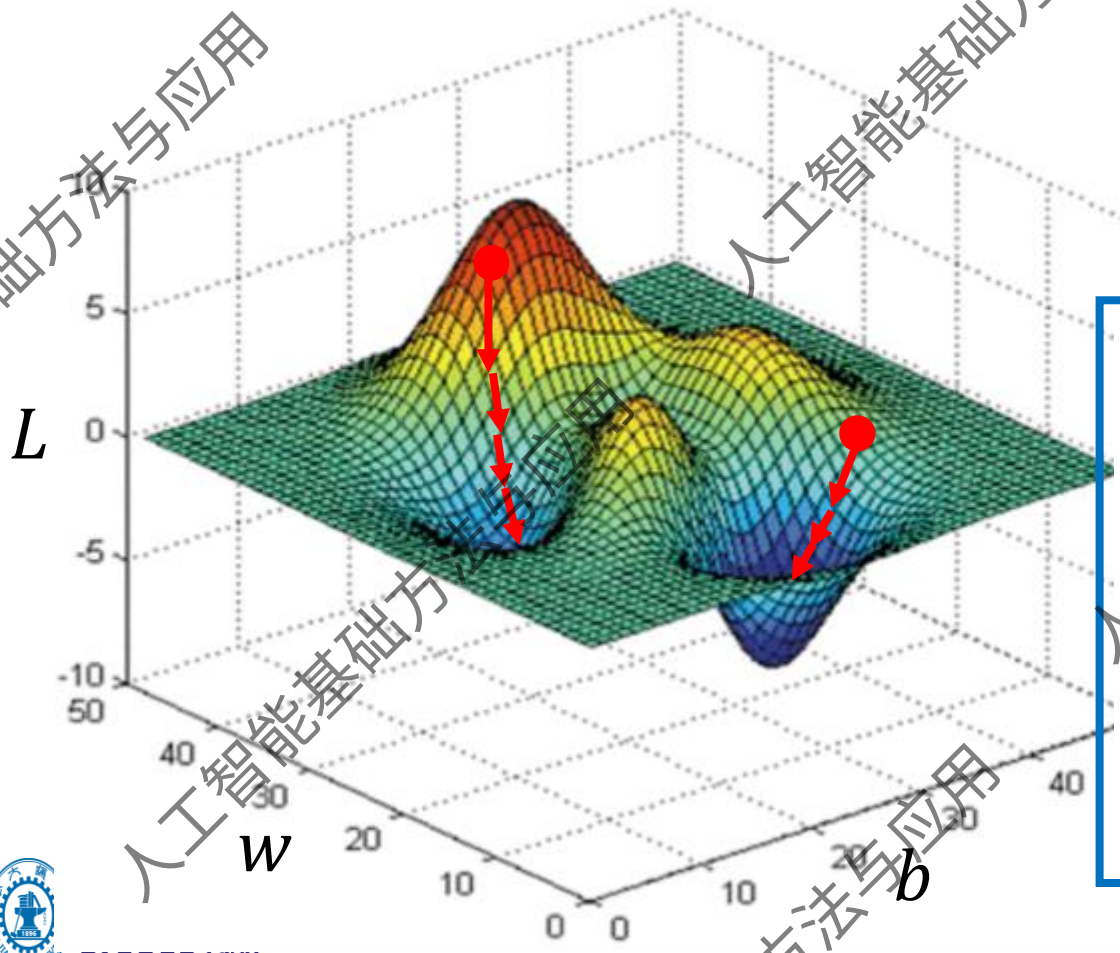
人工智能基础方法与应用

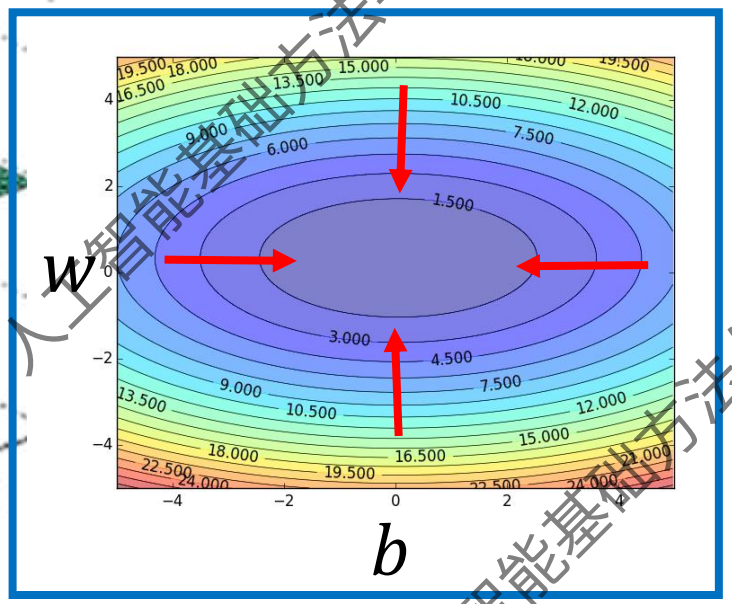
在二维损失地形下降


19.500
18.000
16.500
15.000
13.500
10.500
12.000
9.000
7.500
6.000
1.500
3.000
22.500
24.000
19.500
21.000
4
2
0
−2
−4
−4
−2
0
2
4
w
b
颜色深浅表示损失函数L(w,b)的大小
$(-\mu\,\partial L/\partial b, -\mu\,\partial L/\partial w)$
计算 $\partial L/\partial b, \partial L/\partial w$

神经网络的优化

- 深度神经网络训练通常面对非凸损失地形，因此一般无法保证梯度下降找到全局最优





人工智能基础方法与应用

机器的“学习”没有那么神秘

这就是深度学习中机器“学习”的基本过程……

➡ 甚至 AlphaGo 也使用了类似的方法

人们想象中的 AI……

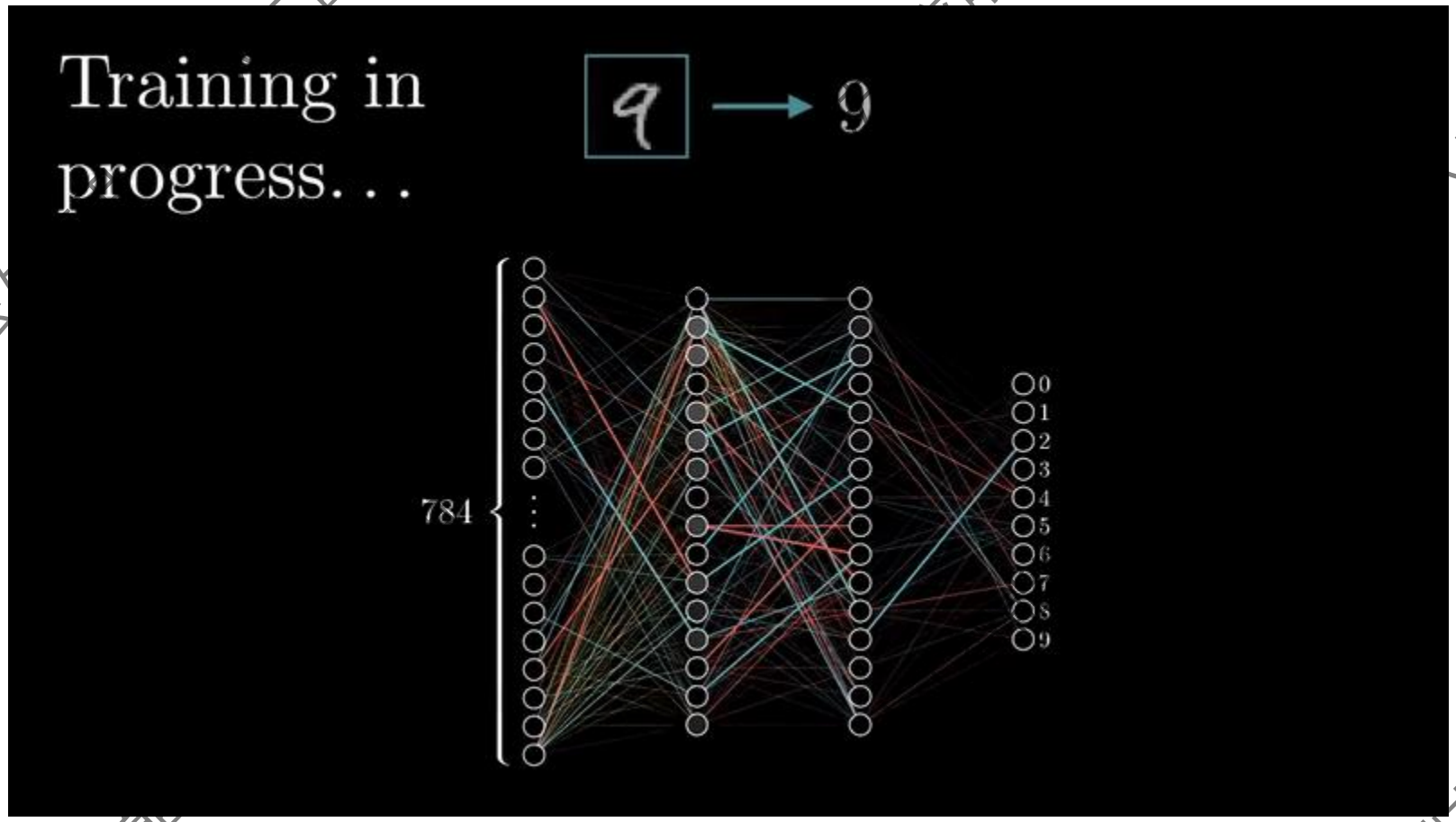


实际上的训练过程 ……



机器学习的“学习”本质上就是不断计算、比较误差并调整参数

人工智能基础方法与应用


Training in progress...
9
784
0
1
2
3
4
5
6
7
8
9

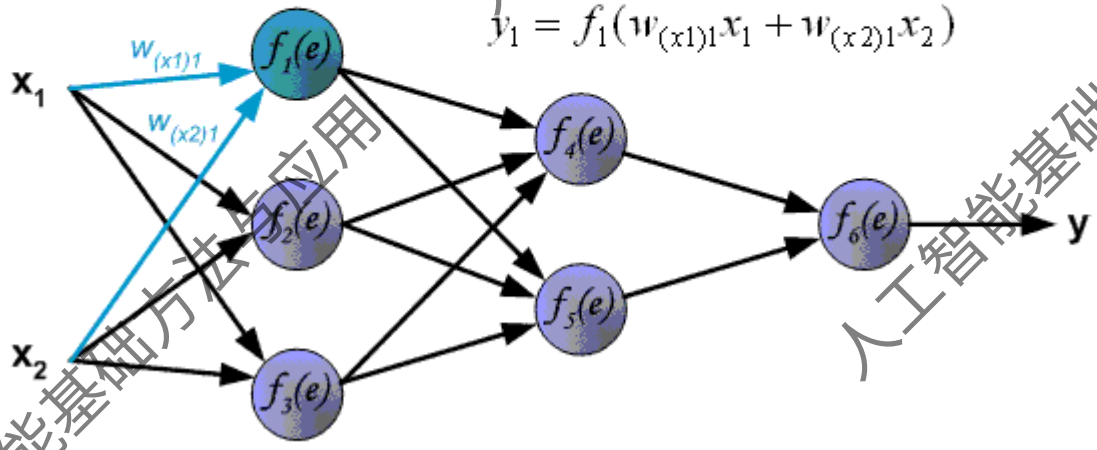
反向传播

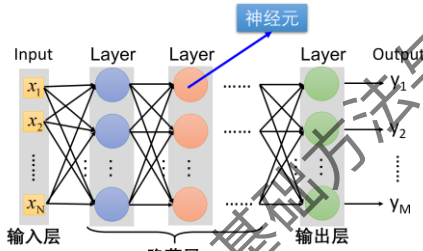
$$\frac{dy}{dx} = \frac{dy}{dz} \cdot \frac{dz}{dx}$$

链式法则

- 利用**链式法则**，从输出层向输入层逐层传播，计算损失函数对各个参数的梯度

FP



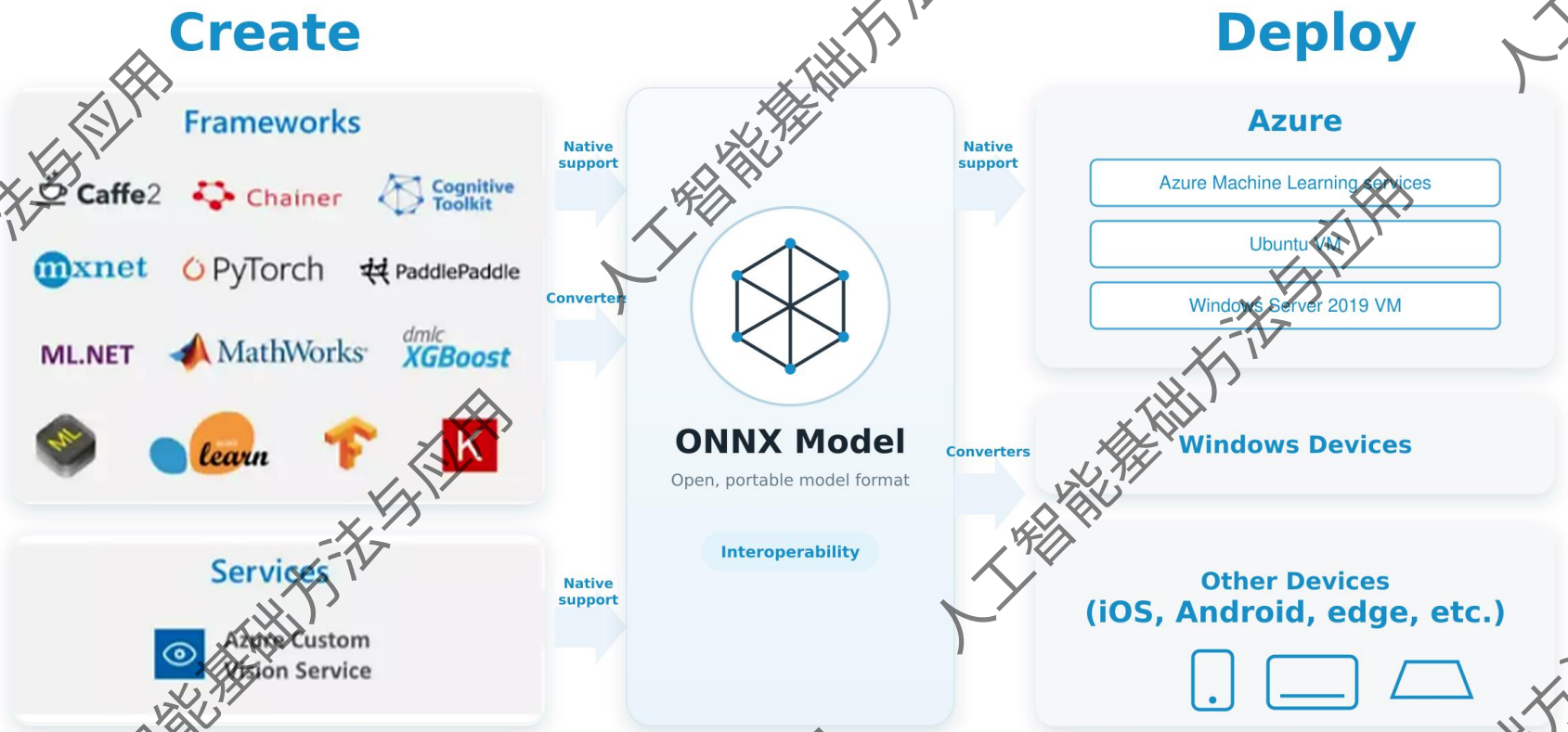


全连接前馈神经网络

人工智能基础方法与应用

反向传播

- 在神经网络中高效计算$\partial L/\partial w$的方式



人工智能基础方法与应用

为什么需要深层结构



人工智能基础方法与应用

越深越好?

语音识别实验

隐藏层数	词错误率(%)
1 X 2k	24.2
2 X 2k	20.4
3 X 2k	18.4
4 X 2k	17.8
5 X 2k	17.2
7 X 2k	17.1

随着网络规模增大、参数增多，词错误率逐渐下降

人工智能基础方法与应用

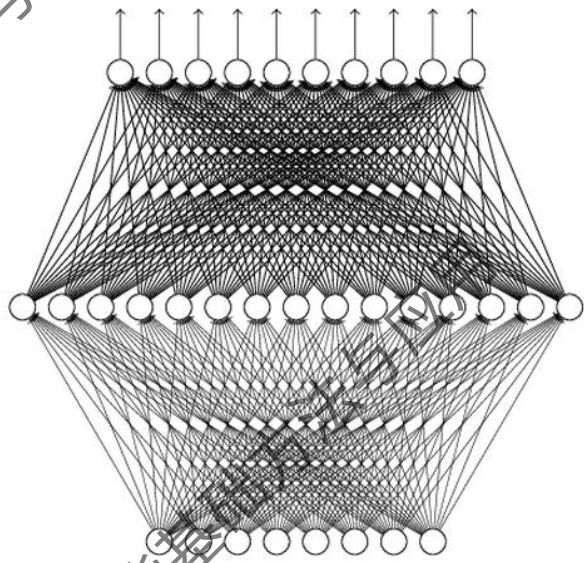
万能逼近定理

任意连续函数f

$$f: R^N \rightarrow R^M$$

都可以由一个只有一层隐藏层的神经网络进行逼近

(前提是：隐藏层中有足够多的神经元)



既然一层隐藏层就可以逼近任意连续函数，为什么还要使用“深”神经网络，而不是“宽”神经网络？

万能逼近定理

隐藏层数	词错误率(%)	隐藏层数	词错误率(%)
1 X 2k	24.2		
2 X 2k	20.4		
3 X 2k	18.4		
4 X 2k	17.8		
5 X 2k	17.2	1 X 3772	22.5
7 X 2k	17.1	1 X 4634	22.6
		1 X 16k	22.1

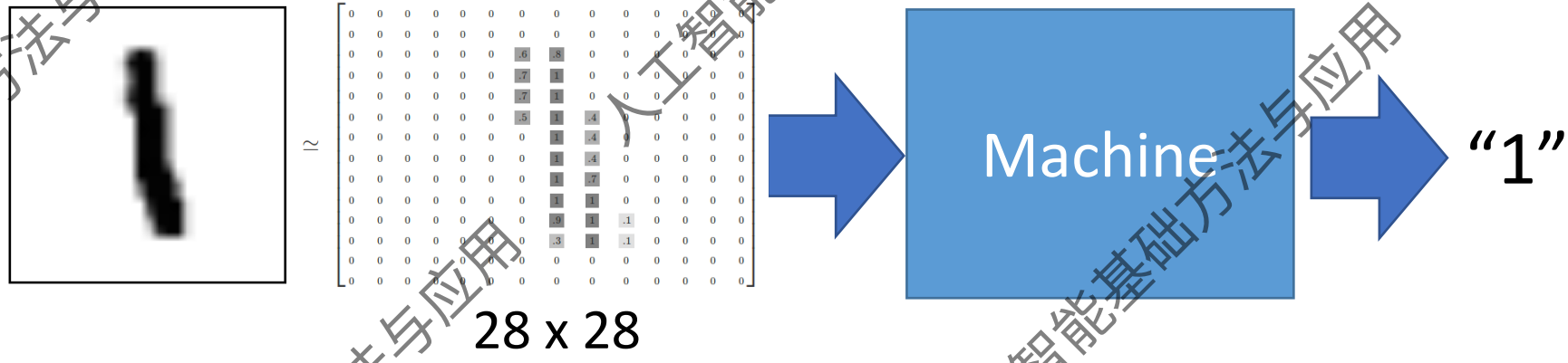
人工智能基础方法与应用

二、完整应用示例

人工智能基础方法与应用

手写数字识别

- MNIST 是最经典的手写数字识别数据集之一，包含 **60,000 张训练图像和 10,000 张测试图像**，每张图像大小为 **28×28 灰度图**，类别为数字 0–9



“Hello world” for deep learning

数据集下载地址：http://yann.lecun.com/exdb/mnist/

PyTorch：深度学习开发框架



人工智能基础方法与应用

PyTorch：深度学习开发框架

- 一个开源深度学习框架，广泛用于人工智能研究和工程开发。它提供了从**张量计算、神经网络构建、自动求导，到模型训练与部署**的一整套工具
- 核心功能
  - **Tensor 张量计算：**类似 NumPy，但可以利用 GPU 加速大规模矩阵运算
  - **自动求导 Autograd：**为反向传播和参数更新提供基础
  - **神经网络模块 torch.nn：**可以方便地搭建全连接网络、CNN、Transformer 等模型
  - **优化器 torch.optim：**提供 SGD、Adam 等常用优化算法
  - **数据处理Dataset/DataLoader：**支持训练数据的读取、批处理和随机打乱

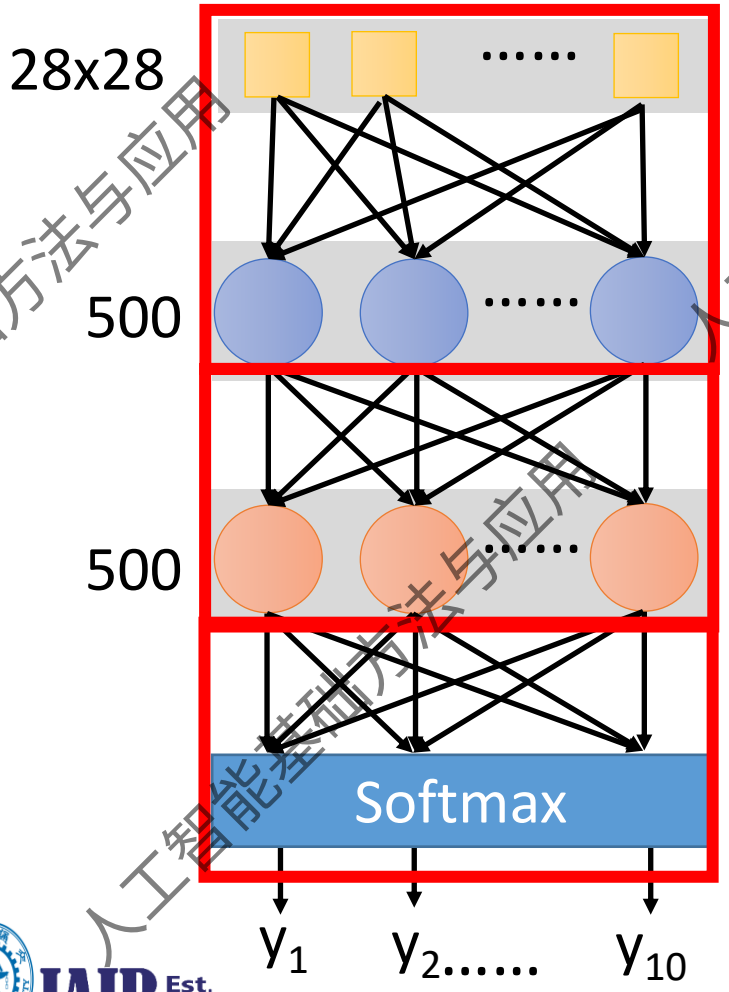
人工智能基础方法与应用

PyTorch

Step 1: define a set of function

Step 2: goodness of function

Step 3: pick the best function

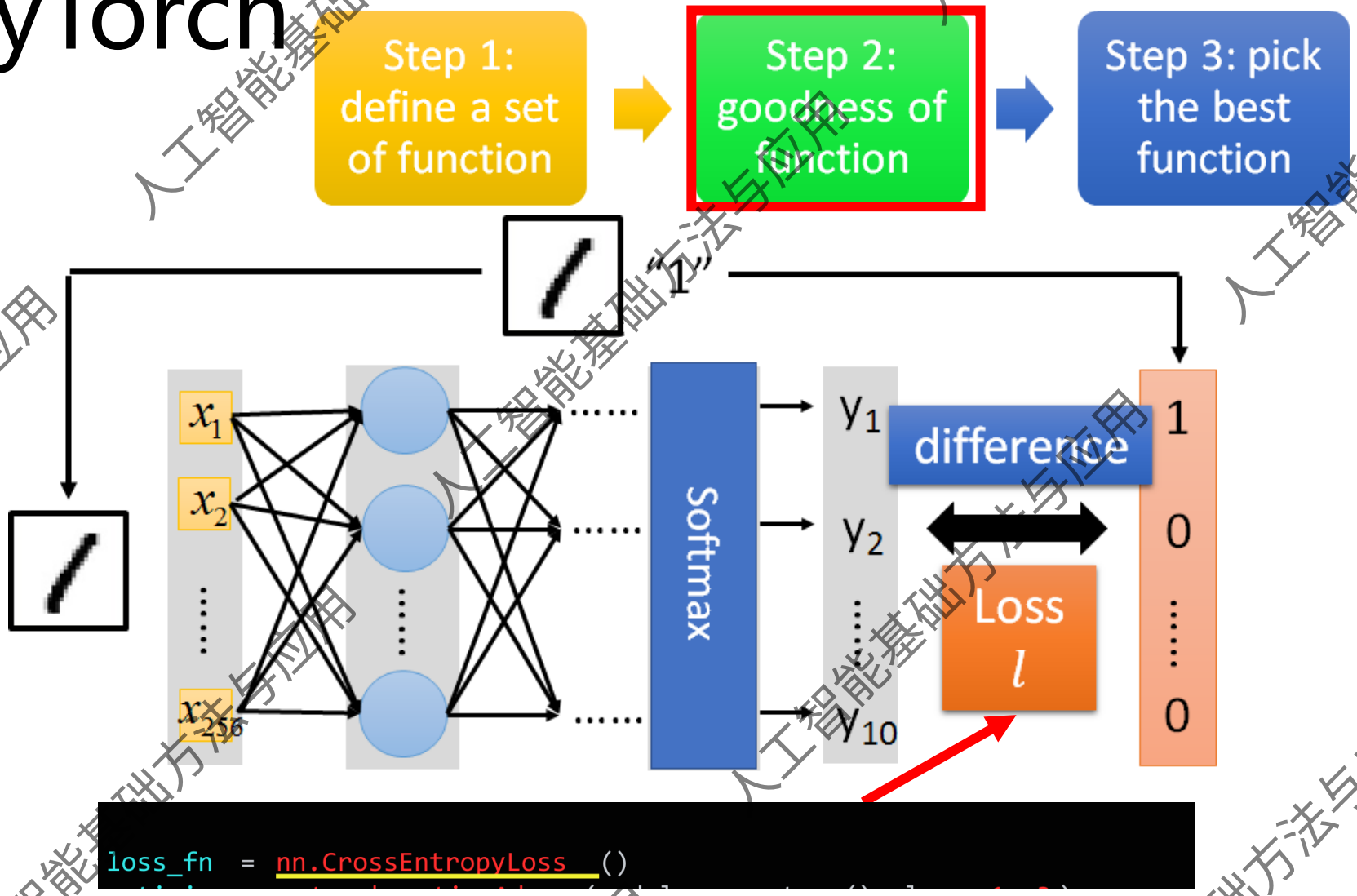



```
model = nn.Sequential (
    nn.Flatten (),
    nn.Linear (28 * 28, 500),
    nn.Sigmoid (),
```

softplus, softsign, relu, tanh, hard_sigmoid, linear

```
    nn.Linear (500, 500),
    nn.Sigmoid (),
    nn.Linear (500, 10)
)
```

PyTorch



人工智能基础方法与应用

PyTorch



Step 3.1: 设定配置

```
loss_fn   = nn.CrossEntropyLoss ()
optimizer   = torch.optim.Adam  (model.parameters(), lr=   1e-3 )
```

SGD, RMSprop, Adagrad, Adadelta, Adam, Adamax, Nadam

Step 3.2: 找到最优的网络参数

```
ds = TensorDataset (x_train ,        y_train ); loader  = DataLoader (ds,  batch size=100  , shuffle= True )
```

训练数据集 (Images)

真实标签 (digits)

数据加载方式

PyTorch

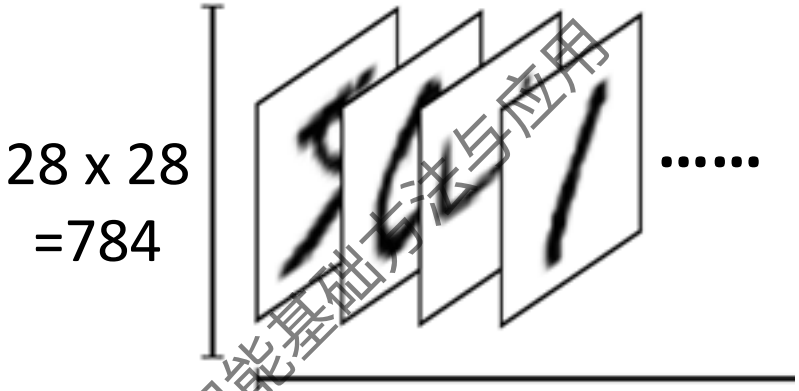


Step 3.2: 找到最优的网络参数

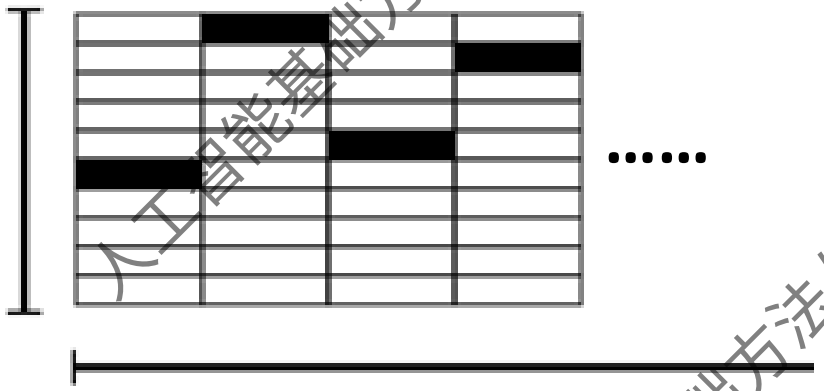
```
ds = TensorDataset (x_train ,        y_train ); loader = DataLoader (ds, batch size=100 , shuffle= True )
```

image tensor

label tensor

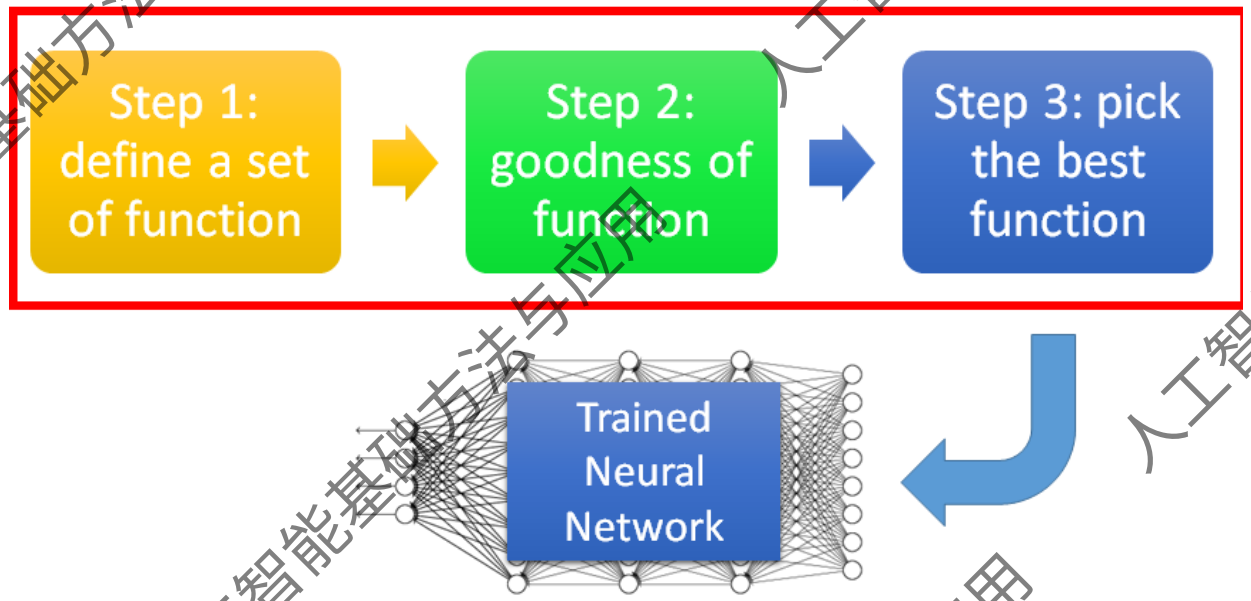


Number of training examples



Number of training examples

PyTorch



保存和加载模型

pytorch.org/tutorials/beginner/saving_loading_models.html

如何使用深度学习网络进行推理:

case 1:

```
model.eval ()
with  torch.inference_mode():
    test_loss, accuracy = test_loop(model, test_loader, loss_fn)
```

case 2:

```
result  = model(x_test).argmax(dim=   1)
```

PyTorch

- 使用 GPU 加速训练
- 方法 1：在 Shell / 命令行中指定 GPU
  - CUDA_VISIBLE_DEVICES=0 python YourCode.py
- 方法 2：在代码中指定 GPU
  - device = torch.device("cuda:0")

    model, x, y = model.to(device), x.to(device), y.to(device)

人工智能基础方法与应用